Lec03：面向对象特征综合应用习题课

Comprehensive application of object-oriented features

课程编号：22WHSE22006（2023版培养计划）

授课对象：23 级软件工程、服务科学与工程专业

主讲教师：辛国栋

Email：xinguodong@163.com



QQ 群：软件构造2025

群　号：311528424

本章内容

- 1. 需求描述与初步的设计
- 2. 设计改进：改善继承引起的设计问题
- 3. 设计改进：分离变化点
- 4. 设计改进：实现复用
- 5. 设计过程复盘与讨论

1. 需求描述与初步的设计

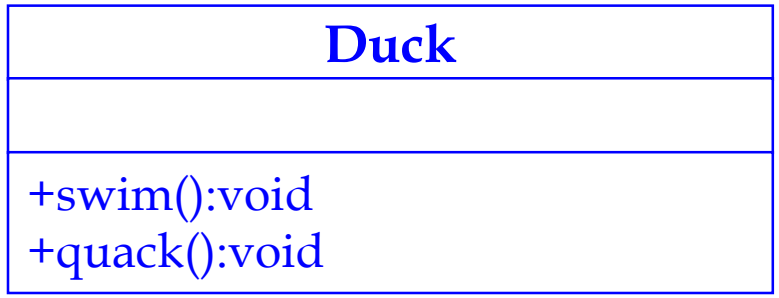
Requirements description and preliminary design

1.1 需求描述

- 某公司要做一套 PC 端游戏：在此游戏中会出现各种类型的鸭子，这些鸭子可以一边游泳戏水 swim，一边呱呱叫 quack
  - 目前已有红头鸭子（red head duck），绿头鸭子（ mallard duck ），未来可能出现其他鸭子
  - 鸭子的 swim 和 quack 可能不同。
  - 界面设计的工作另聘专业团队完成，不用我们来做。
- 所以，我们的任务很明确：设计后端核心代码框架。

1.2 最朴素的设计思想

- 设置一个 Duck 类，所有的鸭子做成 Duck 类的对象。




```
public class Duck {
    public void swim() {
        System.out.println("swim");
    }
    public void quack(){
        System.out.println("quack");
    }
}
```

```
public class DuckTest {
    public static void main(String[] args) {
        Duck readHead = new Duck();
        Duck mallardDuck = new Duck();
        readHead.quack();
        mallardDuck.quack();
        readHead.swim();
        mallardDuck.swim();
    }
}
```

```
D:\tools\jdk-17.0.12\bin\ja
quack
quack
swim
swim
```

测试代码显示，能够形成对象，对象也具有了swim和quack行为

但是这样设计，可行吗?

1.2 最朴素的设计思想

- 弄清什么是对象，什么是类。
  - 当对象具有一致的属性和行为，显然应该是同一个类的实例。
- 例如，某超市的商品管理系统，不同的具体的商品，应该是商品类的对象
  - 属性应该有：商品id，商品名称，商品产地，商品供应商，商品单价，等等
  - 不同的具体的商品，应该是商品类的对象。它们都具有相同的行为和特征时。
- 当对象具有不同的行为和特征时，应该单独设计成类。

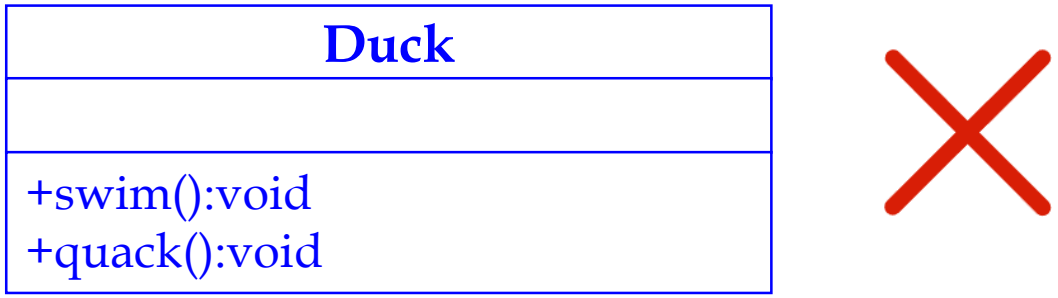
1.2 最朴素的设计思想

甲方的需求

目前已有红头鸭子（red head duck），绿头鸭子（ mallard duck ），未来可能出现其他鸭子

鸭子的 swim 和 quack 可能不同。

我方的设计



- 不同的鸭子的动作可能不同，因此设计成一个类是不合适的。
- 如果需求是：鸭子养殖场的管理系统又会怎样?

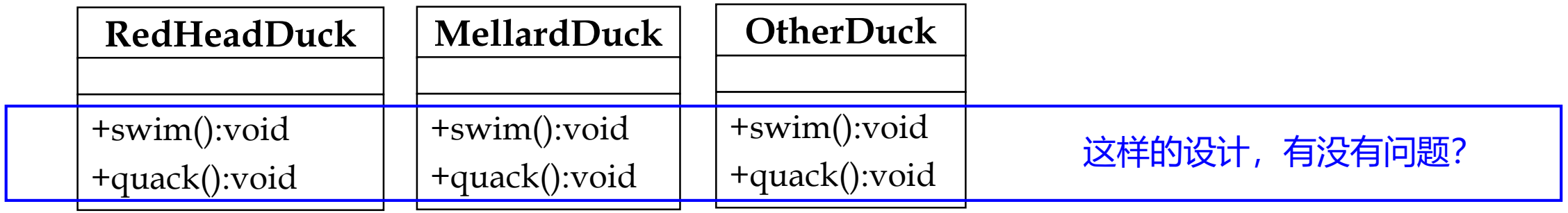
我们就不必关心鸭子的动作，只需要知道鸭子的品种，鸭子编号，鸭子孵出时间等等。

此时，不同的鸭子具有同样的特性和行为，就可以用一个类的不同对象来表示

- **因此，应该设计成不同的鸭子类更合适**

1.2 最朴素的设计思想

- 设计各个类的类名：
  - 红头鸭子：RedHeadDuck类；绿头鸭子：MellardDuck类；其他的鸭子类： OtherDuck
- 设计各个类的属性和操作：
  - 属性：在需求描述中未体现
  - 操作：swim()，quack()；入参无，返回值 void

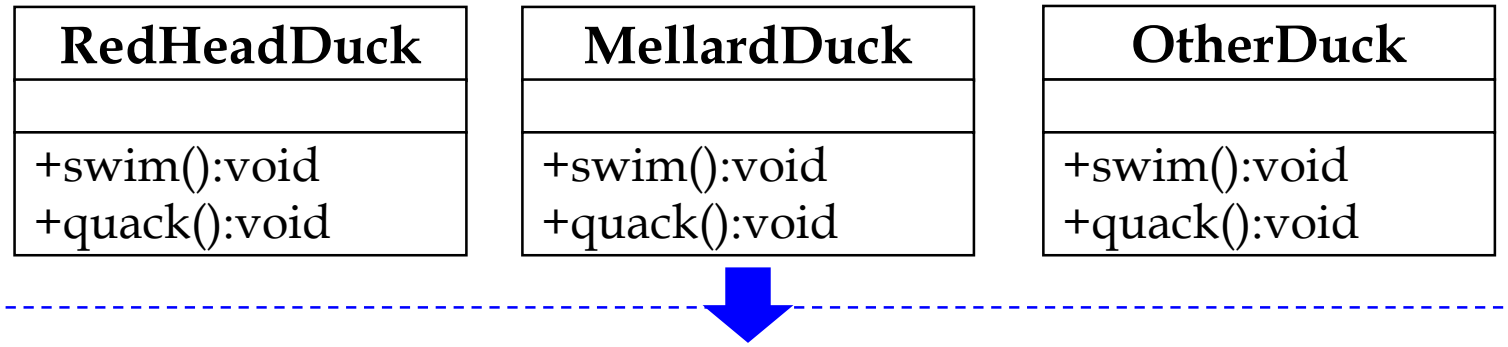


以上设计存在的问题：

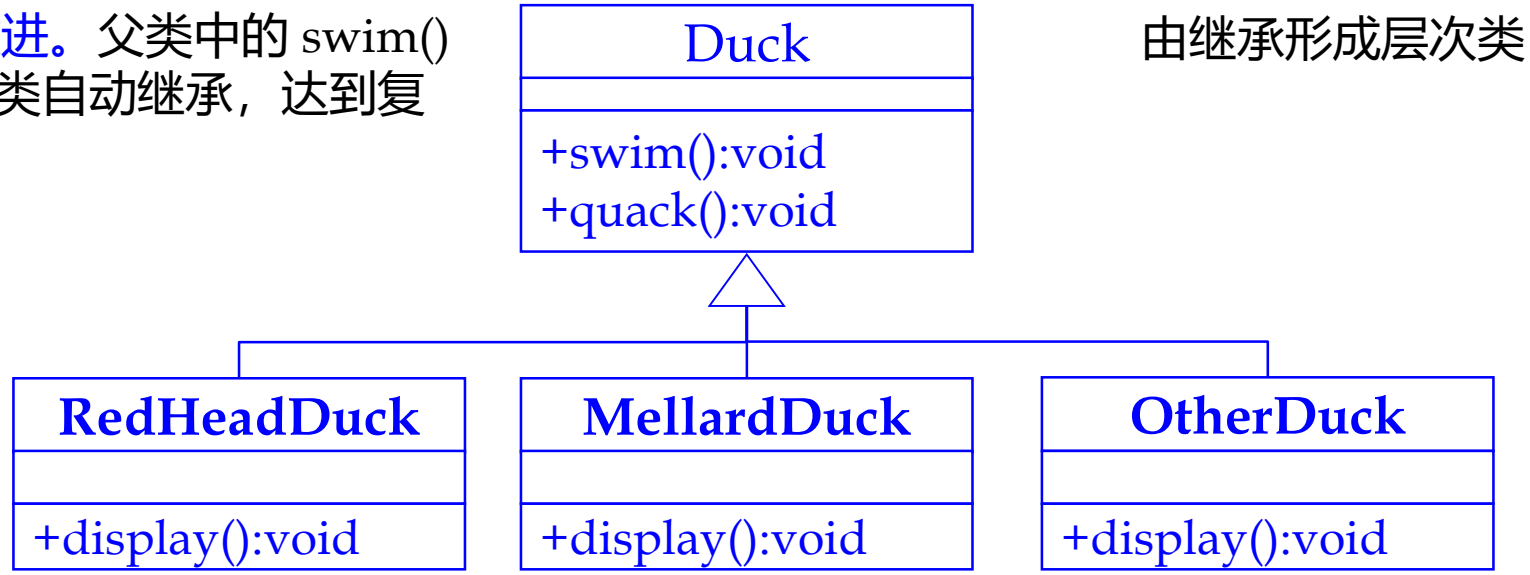
① swim()，quack() 无法复用（Reuse）；

② 如果 swim()，quack() 发生变化，需要在各个类中修改代码，不易维护。

1.3 设计改进：由分散的类形成继承结构



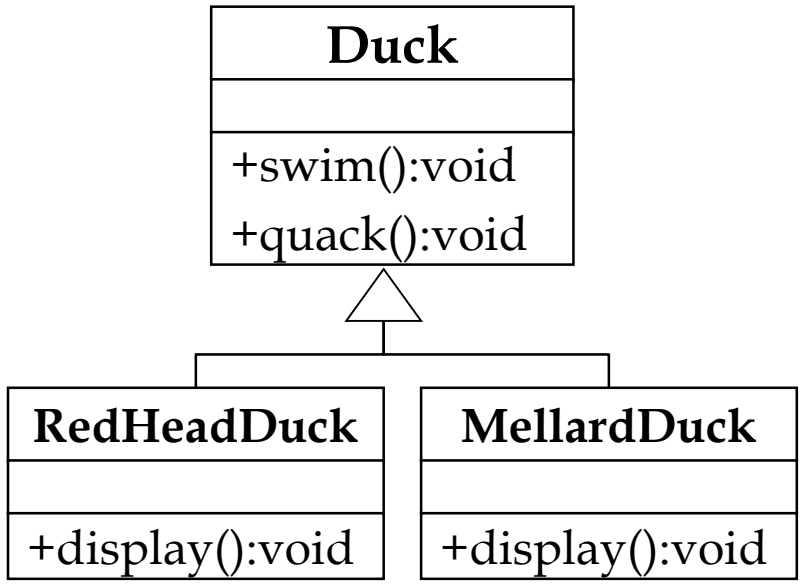
采用继承进行改进。父类中的 swim() 和 quack() 被子类自动继承，达到复用目的；



由继承形成层次类，减少了维护难度。

子类对父类进行了扩展，新增display()，用于展示每种鸭子各自的外观。显然每个鸭子子类的外观不同，其display() 实现代码必然不同，因此不能放到父类复用。

1.3 设计改进：由分散的类形成继承结构




```java
public class Duck {
    public void quack() {
        System.out.println("Duck's quack");
    }
    public void swim() {
        System.out.println("Duck's swim");
    }
}
```

```java
public class MallardDuck extends Duck {
  public void display() {
      System.out.println("MallardDuck's display!");
  }
}
```

```java
public class RedHeadDuck extends Duck {
    public void display() {
        System.out.println("RedheadDuck's display!");
    }
}
```

类图语义很明确：

- Duck 类实现了 swim() 和 quack();
- 子类自动继承 swim() 和 quack();
- 子类具有自己的方法 display();

1.3 设计改进：由分散的类形成继承结构

```
public class DuckTest {
  public static void main(String[] args)  {

    MallardDuck mallardDuck = new MallardDuck();
    RedHeadDuck redHeadDuck = new RedHeadDuck();

    System.out.println("MallardDuck's behavior:");
    mallardDuck.swim();
    mallardDuck.quack();
    mallardDuck.display();

    System.out.println("RedHeadDuck's behavior:");
    redHeadDuck.swim();
    redHeadDuck.quack();
    redHeadDuck.display();
  }
}
```

具体类声明对象，未体现多态，代码僵硬

```
D:\tools\jdk-17.0.12\bin\java.exe "-javaagent:C
MallardDuck's behavior:
Duck's swim
Duck's quack
MallardDuck's display!
RedHeadDuck's behavior:
Duck's swim
Duck's quack
RedheadDuck's display!
```

1.3 设计改进：由分散的类形成继承结构

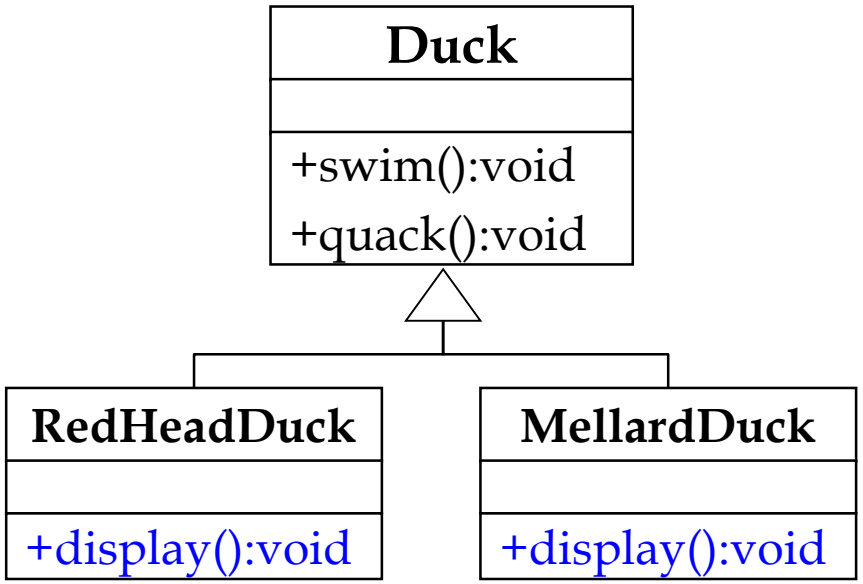
```
public class DuckTest {
  public static void main(String[] args)  {

    Duck duck = new MallardDuck();                    修改为：以父类声明对象，子类实现对象

    System.out.println("MallardDuck's behavior:");
    duck.swim();
    duck.quack();
    duck.display();              编译器报错

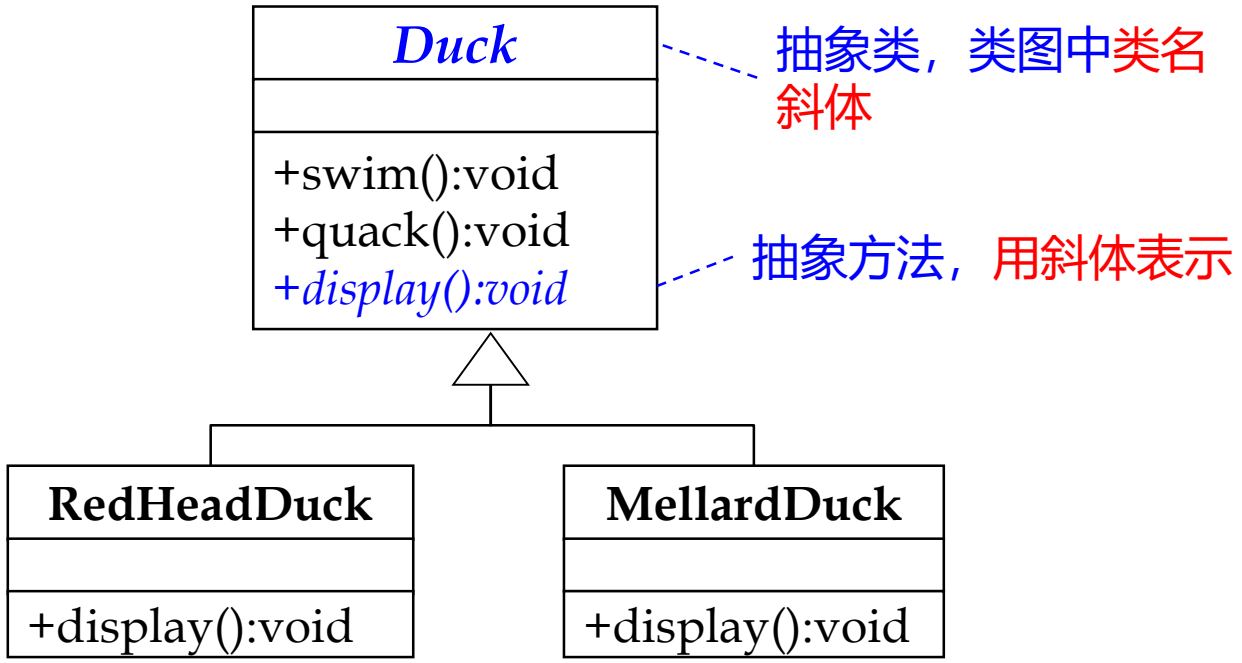
    duck = new RedHeadDuck();                         根据多态，父类对象可以被子类对象替换
    System.out.println("RedHeadDuck's behavior:");
    duck.swim();
    duck.quack();
    duck.display();              编译器报错
  }
}
```

1.4 设计改进：提取抽象方法，超类变成抽象类，实现多态




```
Duck duck = new MallardDuck();
duck.display();    // 编译错误
```

编译错误的原因：Java是静态类型编程语言：变量的数据类型在编译的时候确定；Duck类型中并不存在 display() 因此报错。

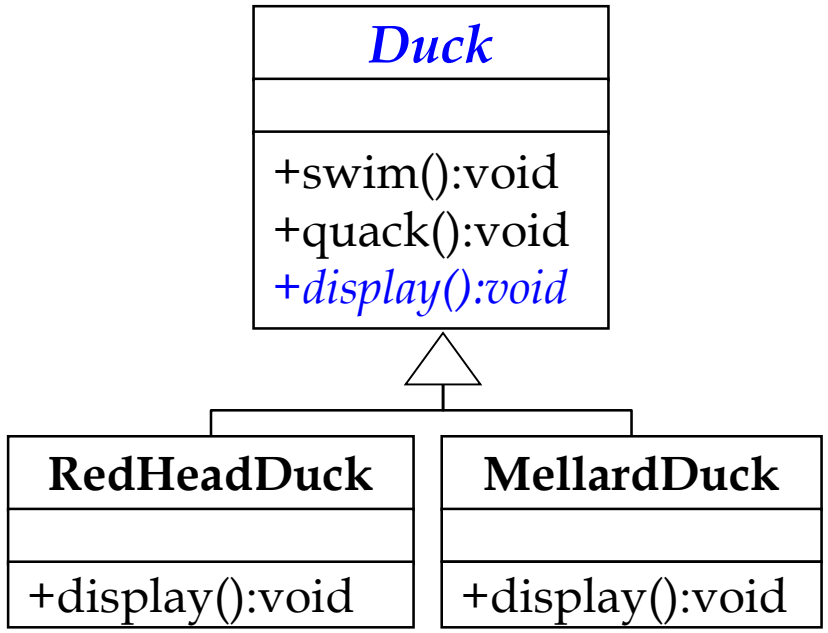


所有子类都具有某方法，并且该方法在不同子类中实现的代码不同，一般做法是将该方法设置为超类的抽象方法。

类中有了抽象方法，该类就变成抽象类，不会直接实例化：

```
Duck duck = new Duck(); // 编译错误
```

1.4 设计改进：提取抽象方法，超类变成抽象类，实现多态




```
public abstract class Duck {
    public void quack() {
        System.out.println("Duck's quack");
    }
    public void swim() {
        System.out.println("Duck's swim");
    }
    public abstract void display();
}
```

```
public class MallardDuck extends Duck {
    @Override  //该注解是jdk覆盖关系校验
    public void display() {
      System.out.println("MallardDuck's display!");
  }
}
```

类图语义：

- 抽象类Duck类实现了swim() 和quack()；子类自动继承;
- 抽象类Duck中具有一个 display抽象方法，子类必须实现超类中的display()，以表现出不同的行为;

RedHeadDuck 代码与 **MallardDuck** 基本一致，自行实现

1.3 设计改进：解决复用与代码分散问题

```
public class DuckTest {
  public static void main(String[] args)  {

    Duck duck = new MallardDuck();                    以父类声明对象，子类实现对象

    System.out.println("MallardDuck's behavior:");
    duck.swim();
    duck.quack();
    duck.display();// 在静态编译时，发现duck对象声明的类型是 Duck，Duck中有display

    duck = new RedHeadDuck();
    System.out.println("RedHeadDuck's behavior:");
    duck.swim();
    duck.quack();
    duck.display();
  }                    // 编译器不再报错
}
```

```
MallardDuck's behavior:
Duck's swim
Duck's quack
MallardDuck's display!
RedHeadDuck's behavior:
Duck's swim
Duck's quack
RedheadDuck's display!
```

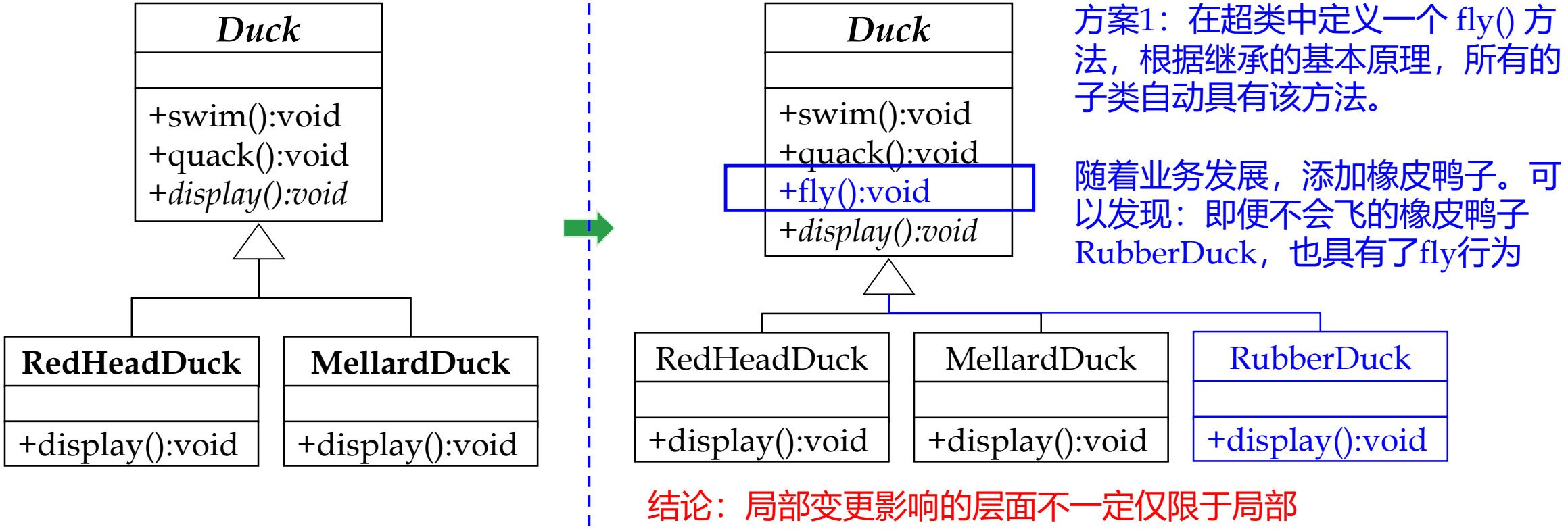
display方法，表现为各自类型的行为，搞定

2. 设计改进：改善继承引起的设计问题

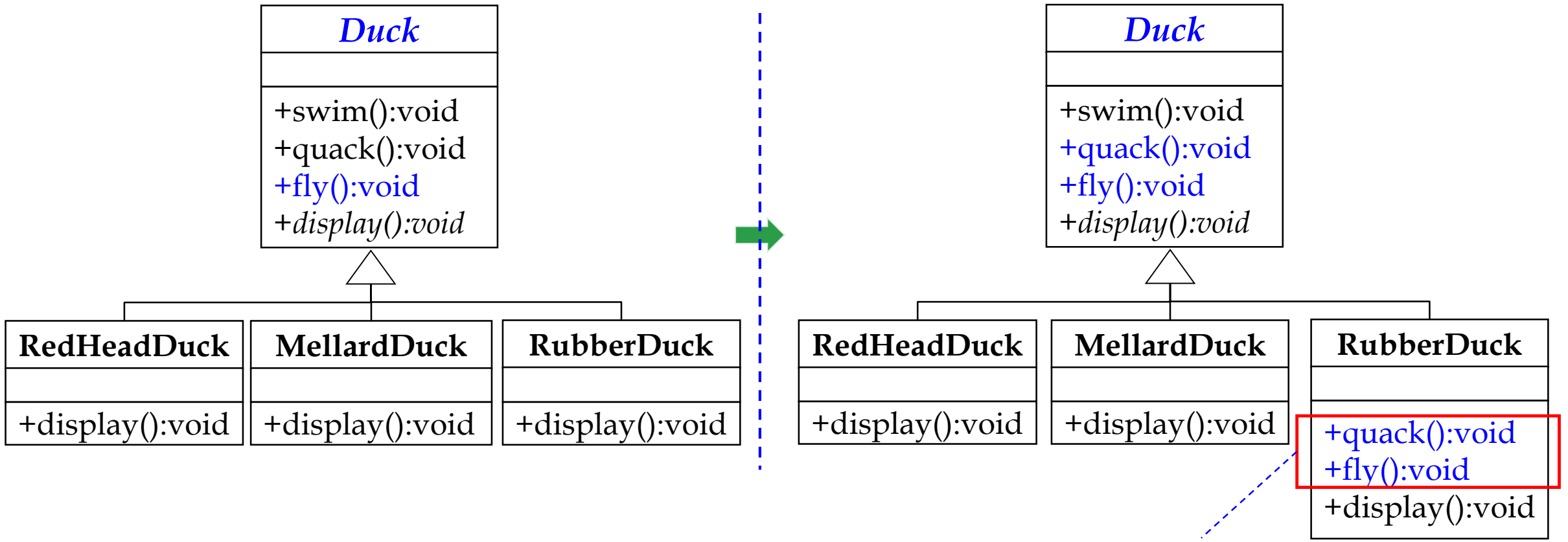
Improve design issues caused by inheritance

2.1 需求变更 – 业务上，让鸭子能飞

- “Change is the only thing that will never change 变化才是唯一不变的永恒”，IT 业界更是如是。随着业务发展和竞争压力加剧，要求游戏中的鸭子会飞。



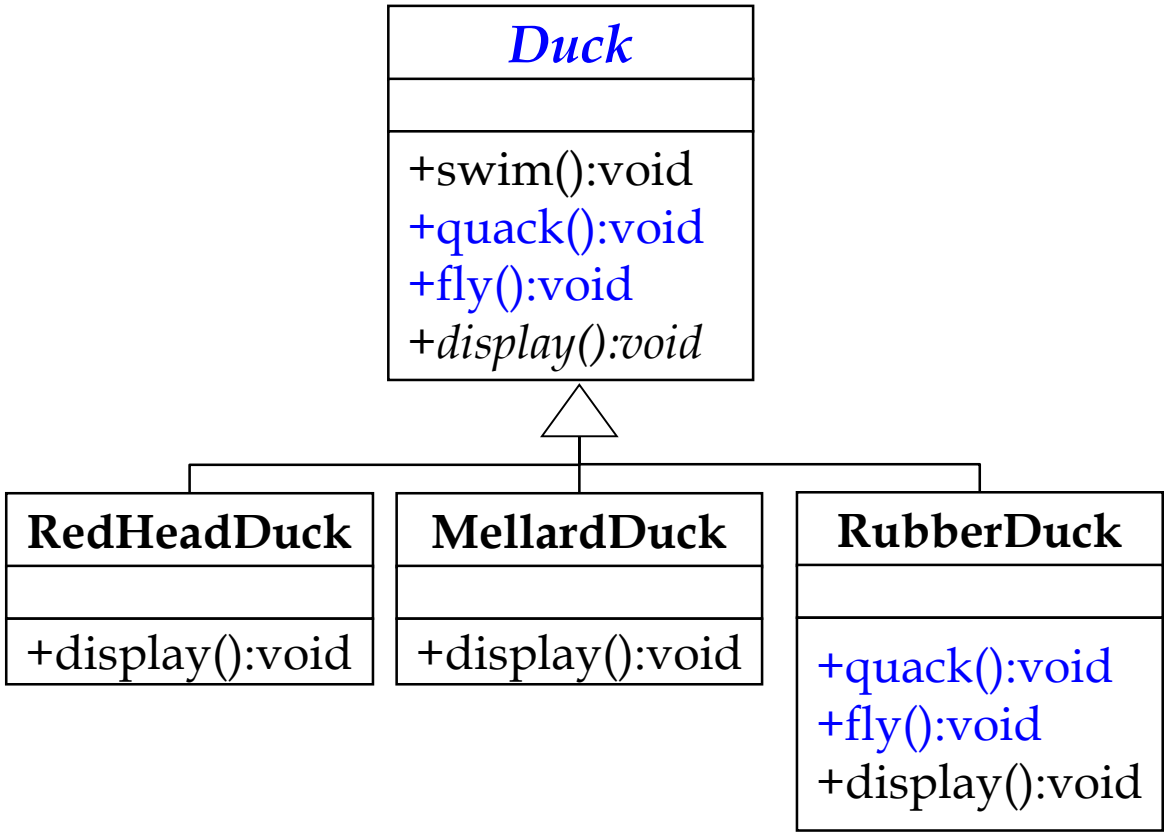
2.2 设计改进：子类通过覆写继承自父类的方法改变自身行为



RubberDuck子类中，除了fly，超类定义的quack也不适用用子类。因此覆盖这两个方法。

为了让案例具有代表性，假定所有子类的 swim 行为都一致，通过继承实现复用即可。
但是，fly() 和 quack() 可能不同

2.2 设计改进：子类通过覆写继承自父类的方法改变自身行为



quack()、fly() 两个方法子类需要根据自己的情况重写

```
public abstract class Duck {
    public void quack() {
        System.out.println("Duck's quack");
    }
    public void swim() {
        System.out.println("Duck's swim");
    }
    public void fly() {
        System.out.println("Duck's fly");
    }
    public abstract void display();
}
```

```
public class RubberDuck extends Duck {
  public void quack() { // 覆盖，超类定义不适合子类
    System.out.println("RubberDuck's quack!");
  }
  public void fly() { // 覆盖
    System.out.println("RubberDuck's fly!");
  }
  public void display() {
    System.out.println("RubberDuck's display!");
  }
}
```

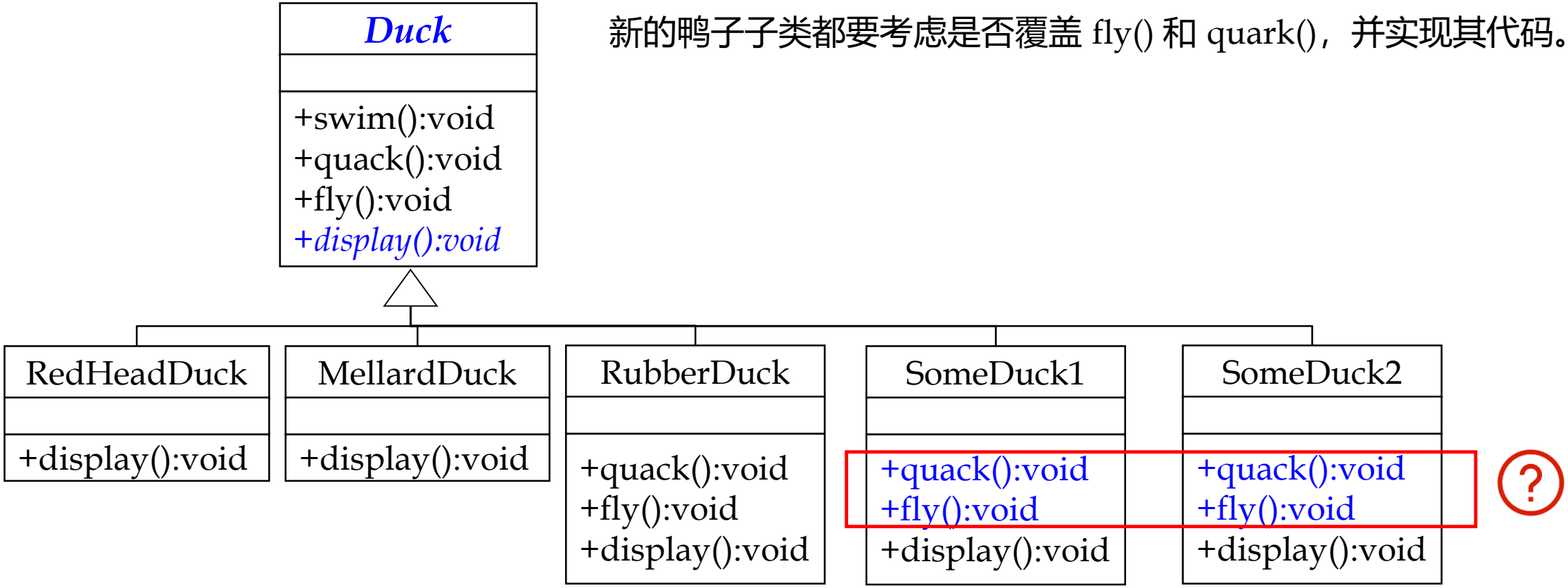
2.2 设计改进：子类通过覆写继承自父类的方法改变自身行为

```
public class DuckTest {
    public static void main(String[] args) {
        Duck duck = new MallardDuck();
        System.out.println("MallardDuck's behavior:");
        duck.swim();
        duck.quack();
        duck.display();
        duck.fly();
        System.out.println("======================");
        duck = new RubberDuck();
        System.out.println("RubberDuck's behavior:");
        duck.swim();
        duck.quack();
        duck.display();
        duck.fly();
    }
}
```

```
MallardDuck's behavior:
Duck's swim
Duck's quack
MallardDuck's display!
Duck's fly
======================
RubberDuck's behavior:
Duck's swim
RubberDuck's quack!
RubberDuck's display!
RubberDuck's fly!
```

子类具备了与父类不同的行为

2.3 通过覆写改变子类的行为存在的问题



新的鸭子子类都要考虑是否覆盖 fly() 和 quark()，并实现其代码。

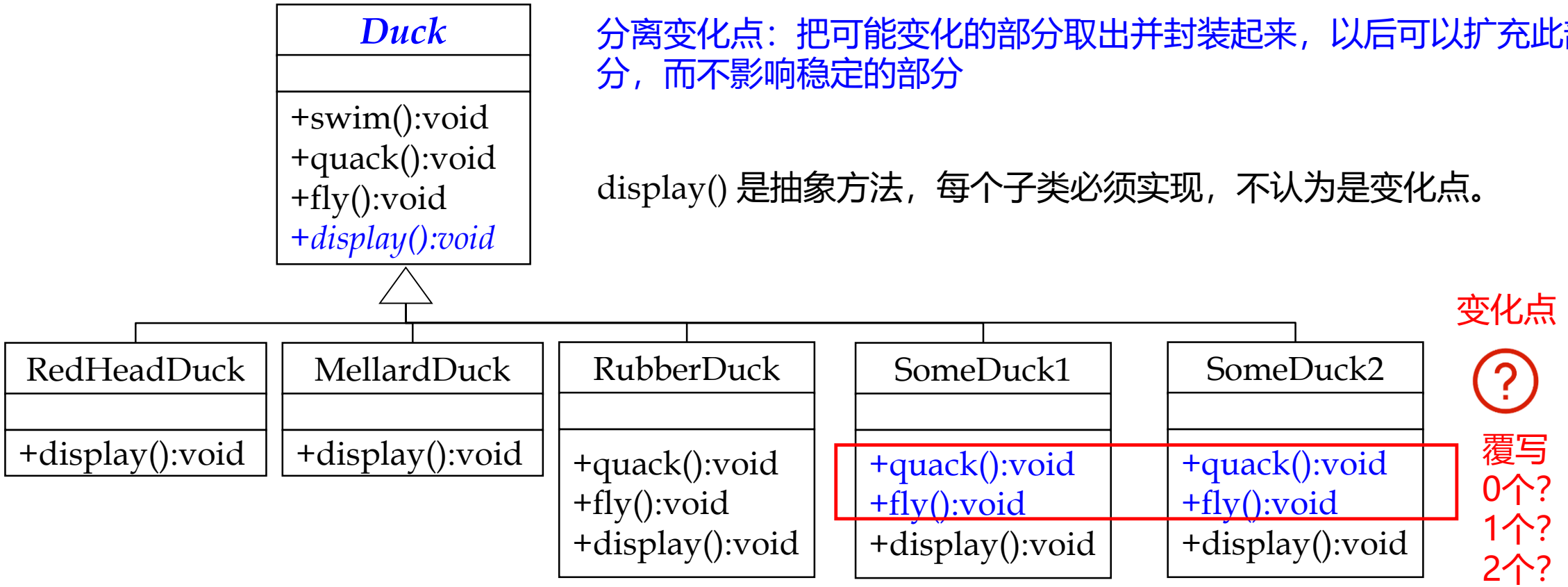
- 分析每个子类：有的代码是稳定的，不发生变化，例如 swim()；
- 有的代码可能发生变化，不稳定，例如 quack() 和 fly() 。

稳定的代码和不稳定的代码放在一起，真的合适吗？

3. 设计改进：分离变化点

Separation of change points

3.1 设计改进：分离变化点的思想



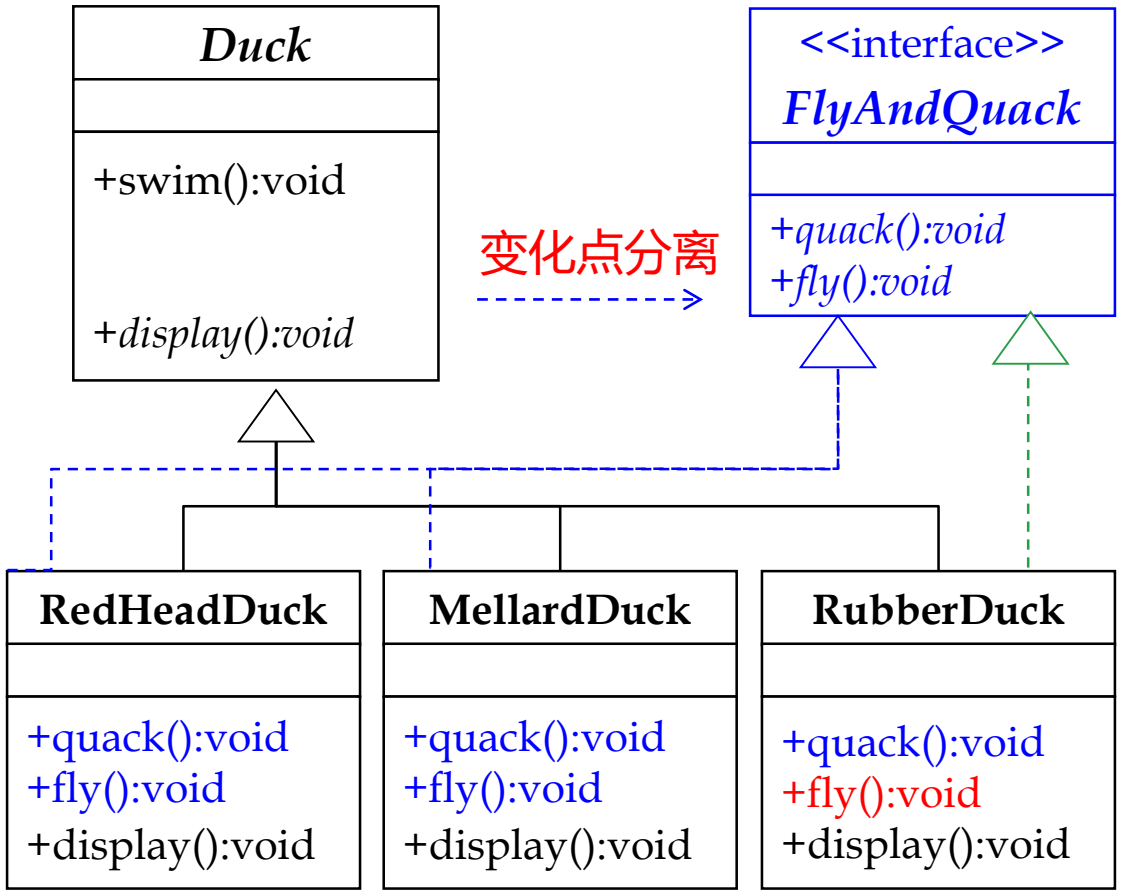
分离变化点：把可能变化的部分取出并封装起来，以后可以扩充此部分，而不影响稳定的部分

display() 是抽象方法，每个子类必须实现，不认为是变化点。

很显然 quack() 和 fly() 是变化点。剩下的问题是，如何分离变化点?

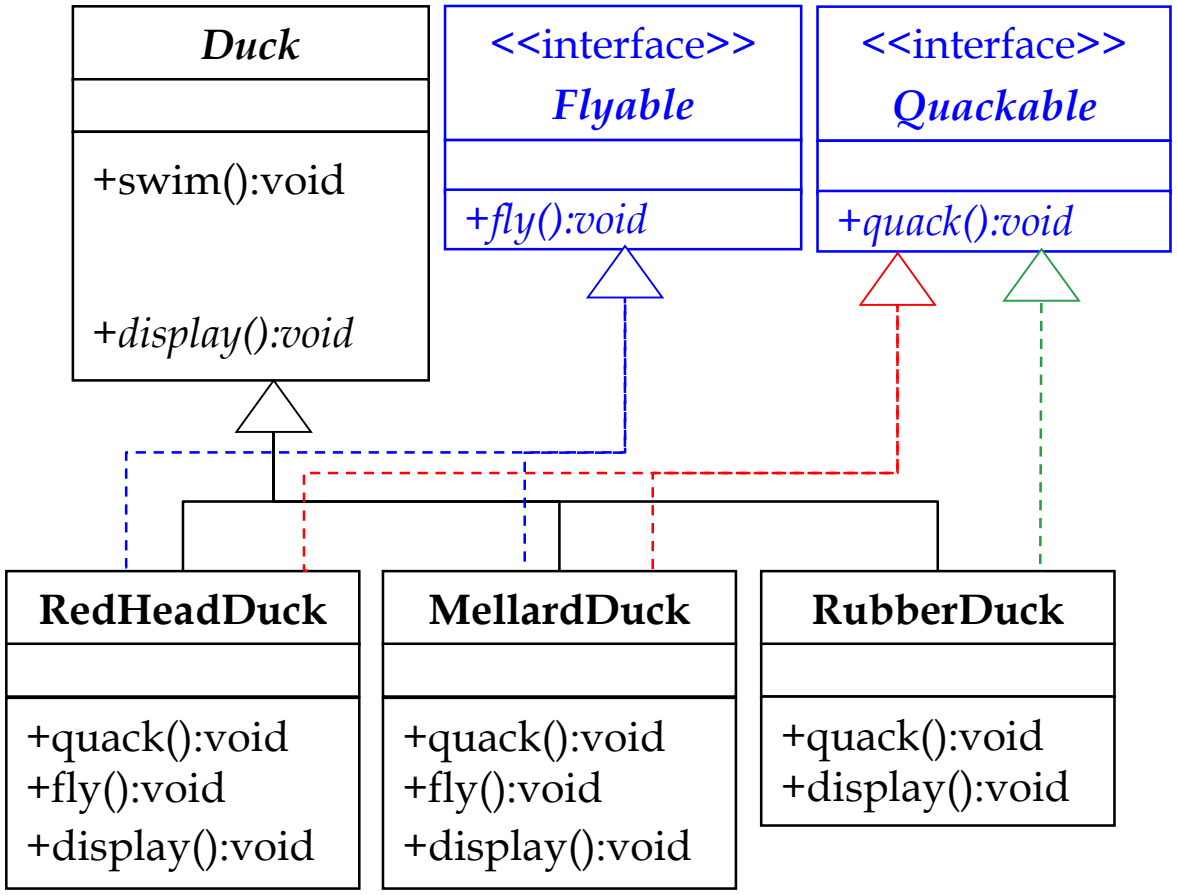
Java 代码必须包裹在 类、接口之中。因此考虑 quack(), fly() 代码放到哪种类型中合适。

3.2 如何分离变化点 – 采用接口类分离变化点



方案1：quack() 和 fly() 分离到一个接口

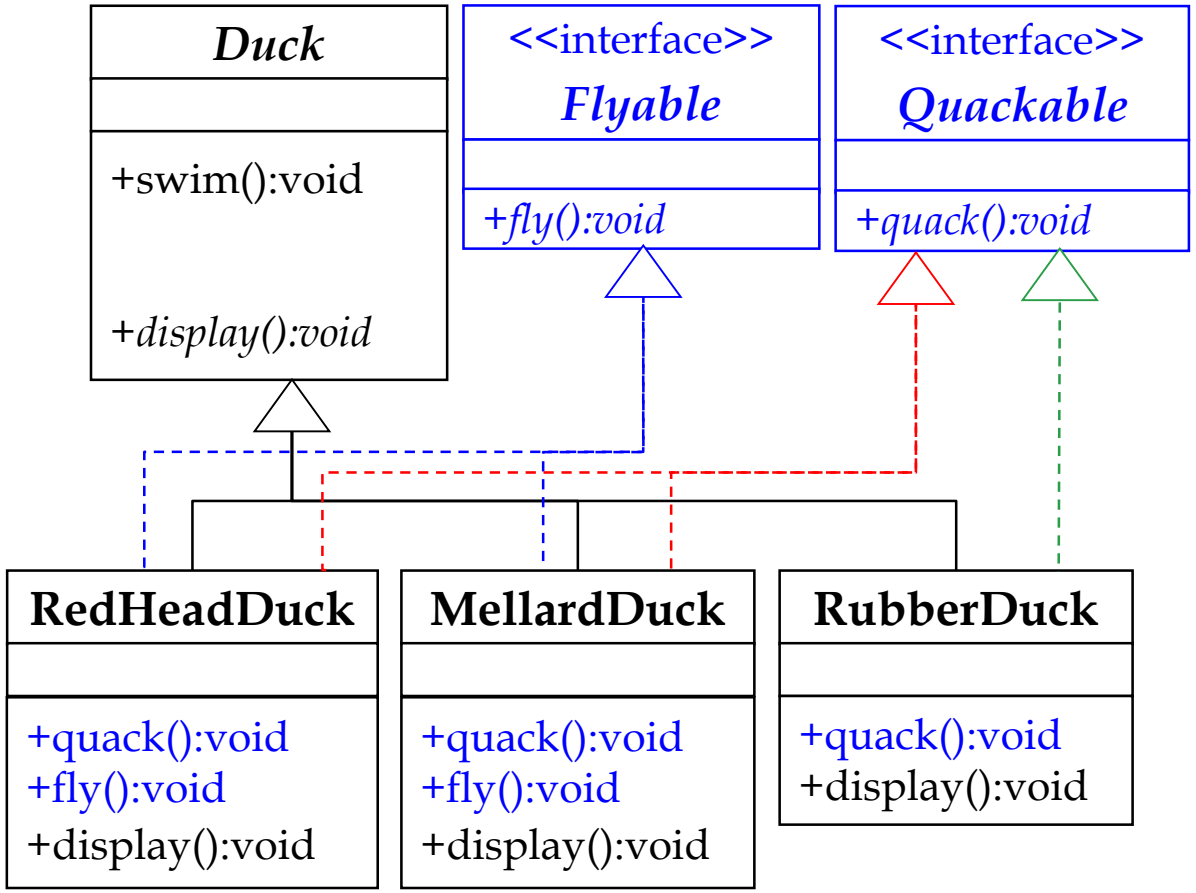
实现 FlyAndQuack 接口类的子类必须 会飞 + 会叫，无论是否真正具有这两种行为



方案2：quack() 和 fly() 分离到不同的接口

通过实现不同接口类满足：有的会飞，有的会叫，有的会飞+会叫；

3.2 如何分离变化点 – 采用接口类分离变化点

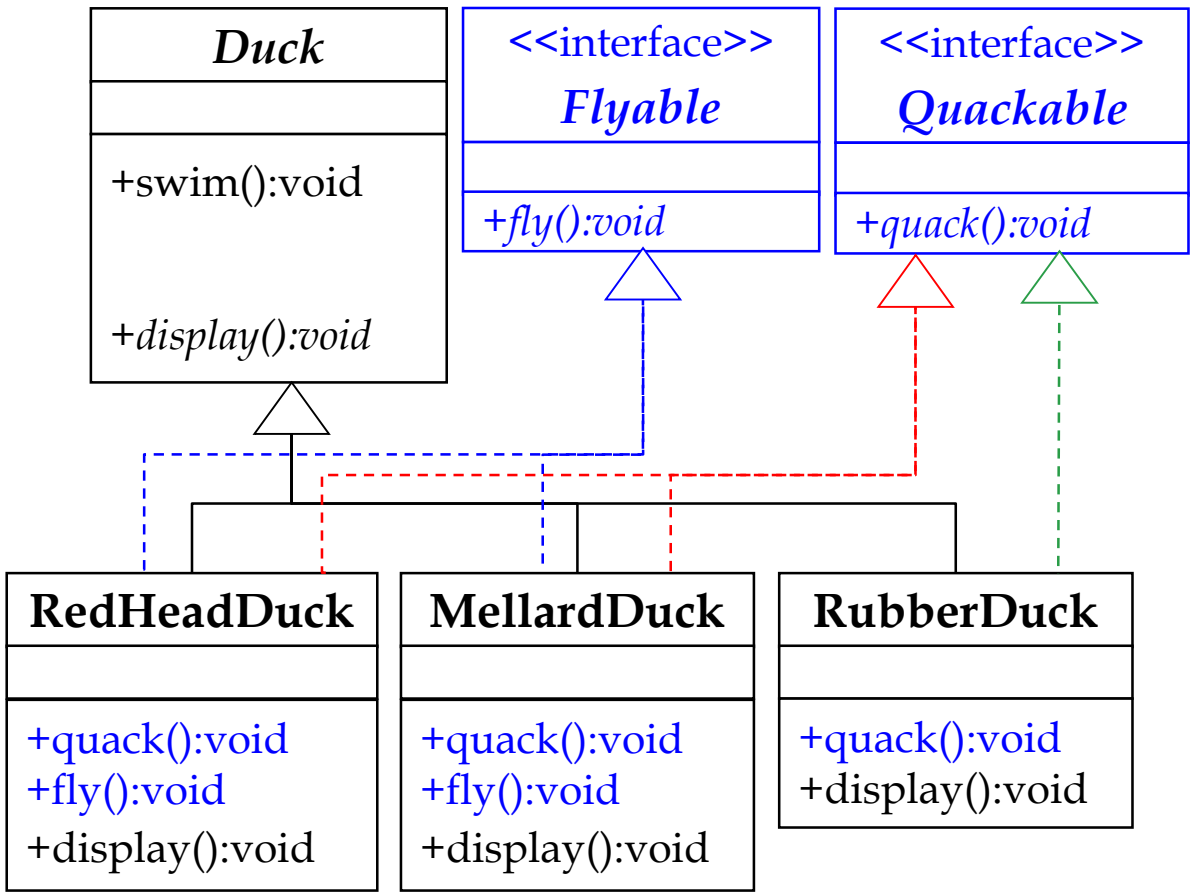



```
public abstract class Duck {
    public void swim() {
        System.out.println("Duck's swim");
    }
    public abstract void display();
}
```

```
public interface Flyable {
    void fly();
}
```

```
public interface Quackable {
    void quack();
}
```

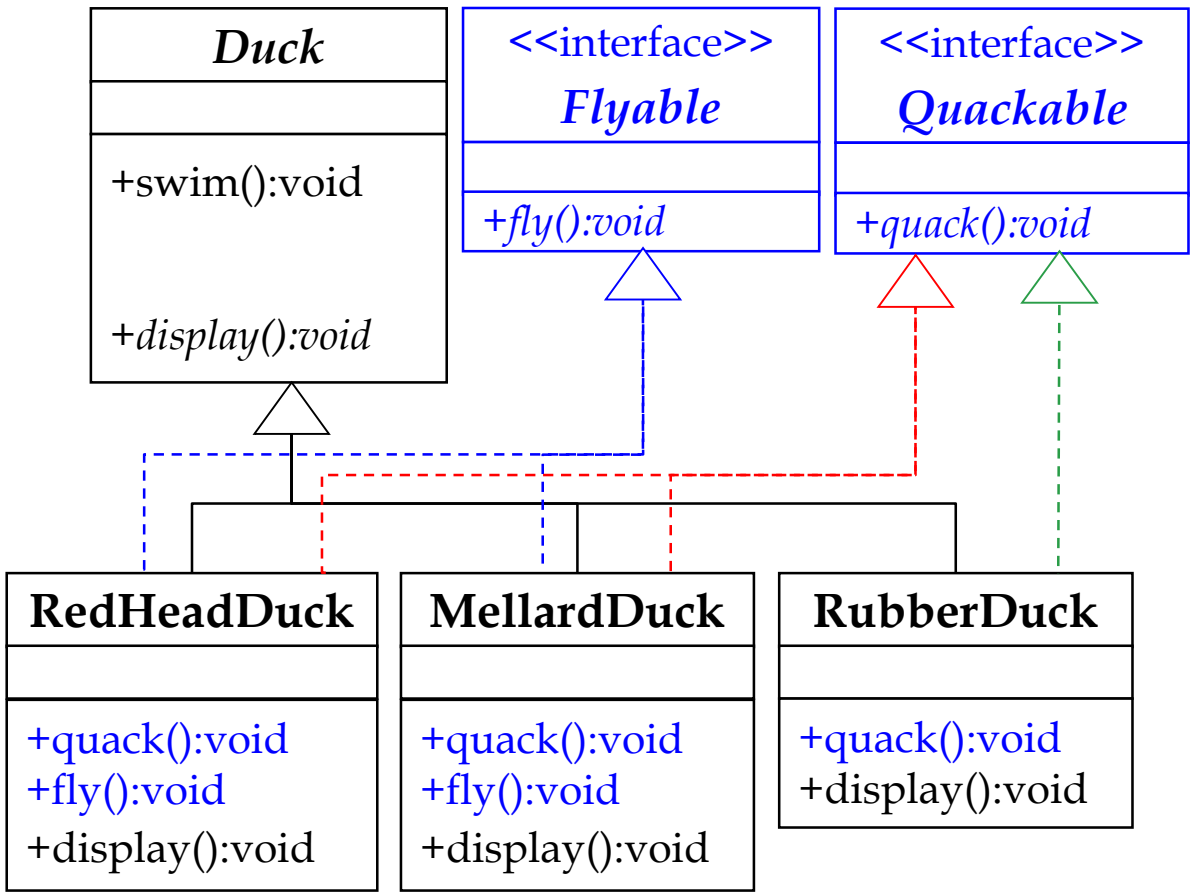
类图语义：所有的duck子类都会swim，也有展示外观的方法。此外，RedHeadDuck和MellardDuck 会飞会叫，而RubberDuck只会叫。



类名之后先写继承，后写实现，如果实现多个接口，接口之间用半角逗号分隔

```java
public class MallardDuck extends Duck implements
Quackable, Flyable {
    @Override
    public void fly() {
        System.out.println("Mallard's fly");
    }
    @Override
    public void quack() {
        System.out.println("Mallard's quack");
    }
    @Override
    public void display() {
        System.out.println("Mallard's display!");
    }
}
```

```java
public class RubberDuck extends Duck implements
Quackable {
  @Override
  public void quack() { // 覆盖，超类定义不适合子类
     System.out.println("Rubber's quack!");
  }
  @Override
  public void display() {
    System.out.println("Rubber's display!");
  }
}
```




```
public class DuckTest {
    public static void main(String[] args) {

        MallardDuck mduck = new MallardDuck();
        System.out.println("Mallard's behavior:");
        mduck.swim();
        mduck.quack();
        mduck.display();
        mduck.fly();
        System.out.println("============");

        RubberDuck rduck = new RubberDuck();
        System.out.println("Rubber's behavior:");
        rduck.swim();
        rduck.quack();
        rduck.display();
    }
}
```

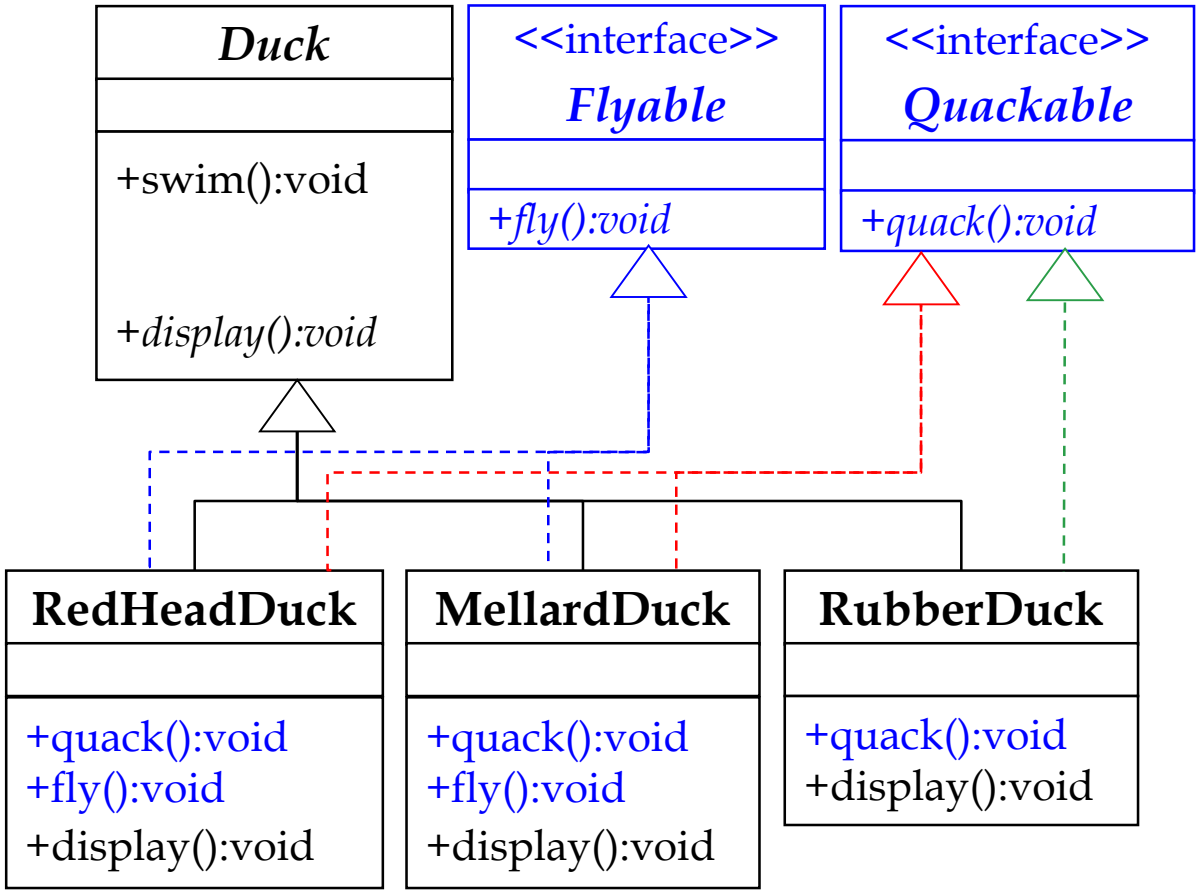
该设计的优点：（1）分离了变化点，设计思路清晰；（2）容易维护，可以任意添加子类，不影响已有代码

该设计的缺点：（1）每个类的 fly(), quack() 的实现，还是得自己写，无法复用；（2）丧失了多态特性，无法用超类声明对象

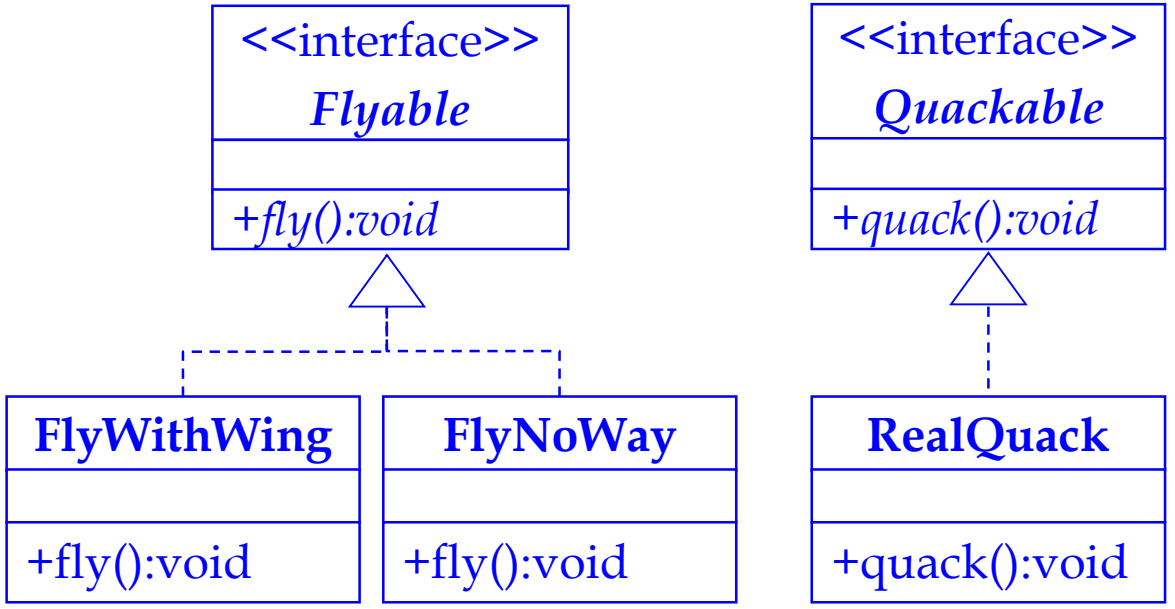
4. 设计改进：实现复用

Achieve code reuse

4.1 无法复用的原因分析及复用的解决方案

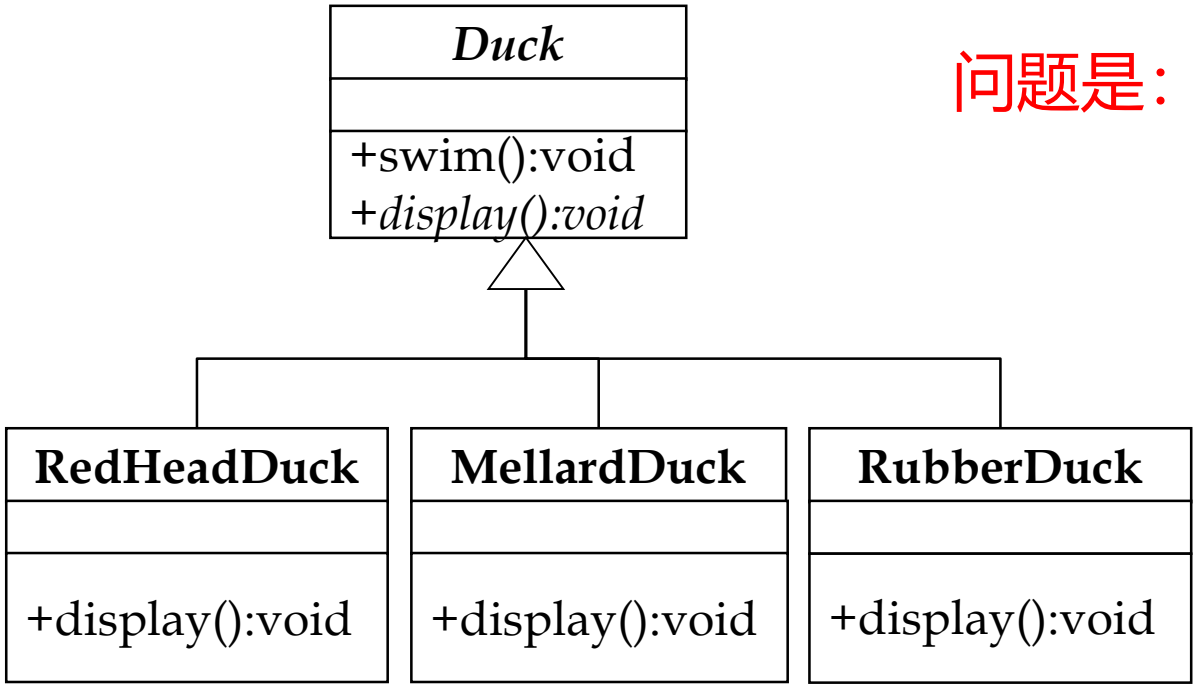


显然，应该让接口有实现类。在接口的实现类中才有已经写好的代码，才有复用的可能。

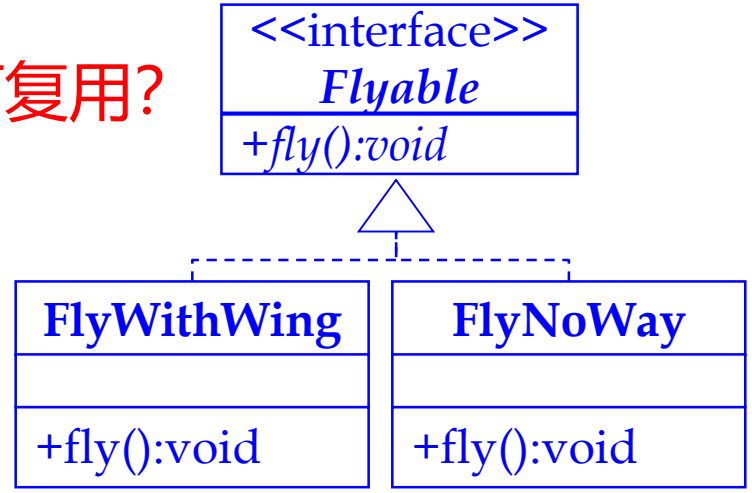


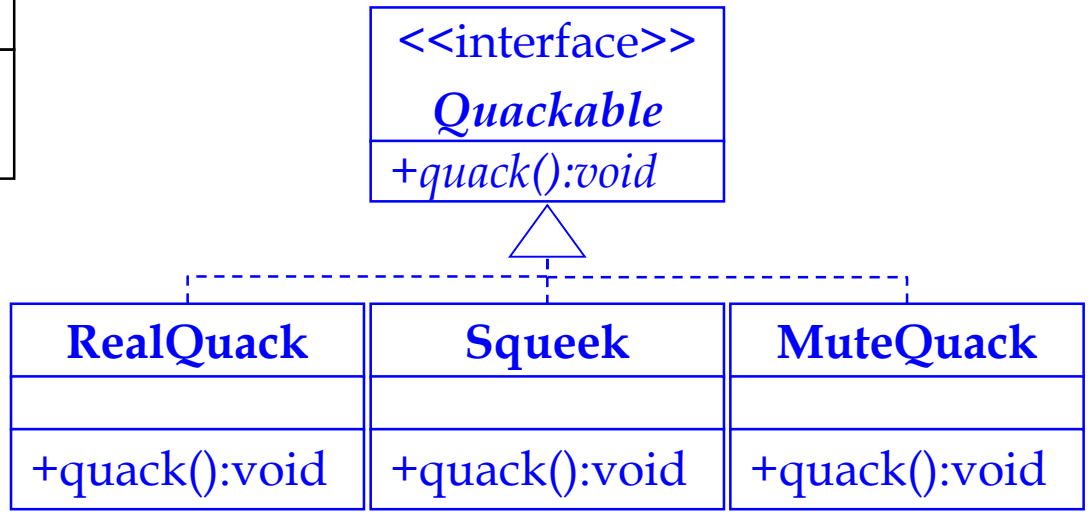
既然该设计无法复用代码，想办法让其复用即可。如何让一个接口中的代码复用?

4.2 审视变化点与稳定部分形成的类结构



问题是：如何复用？

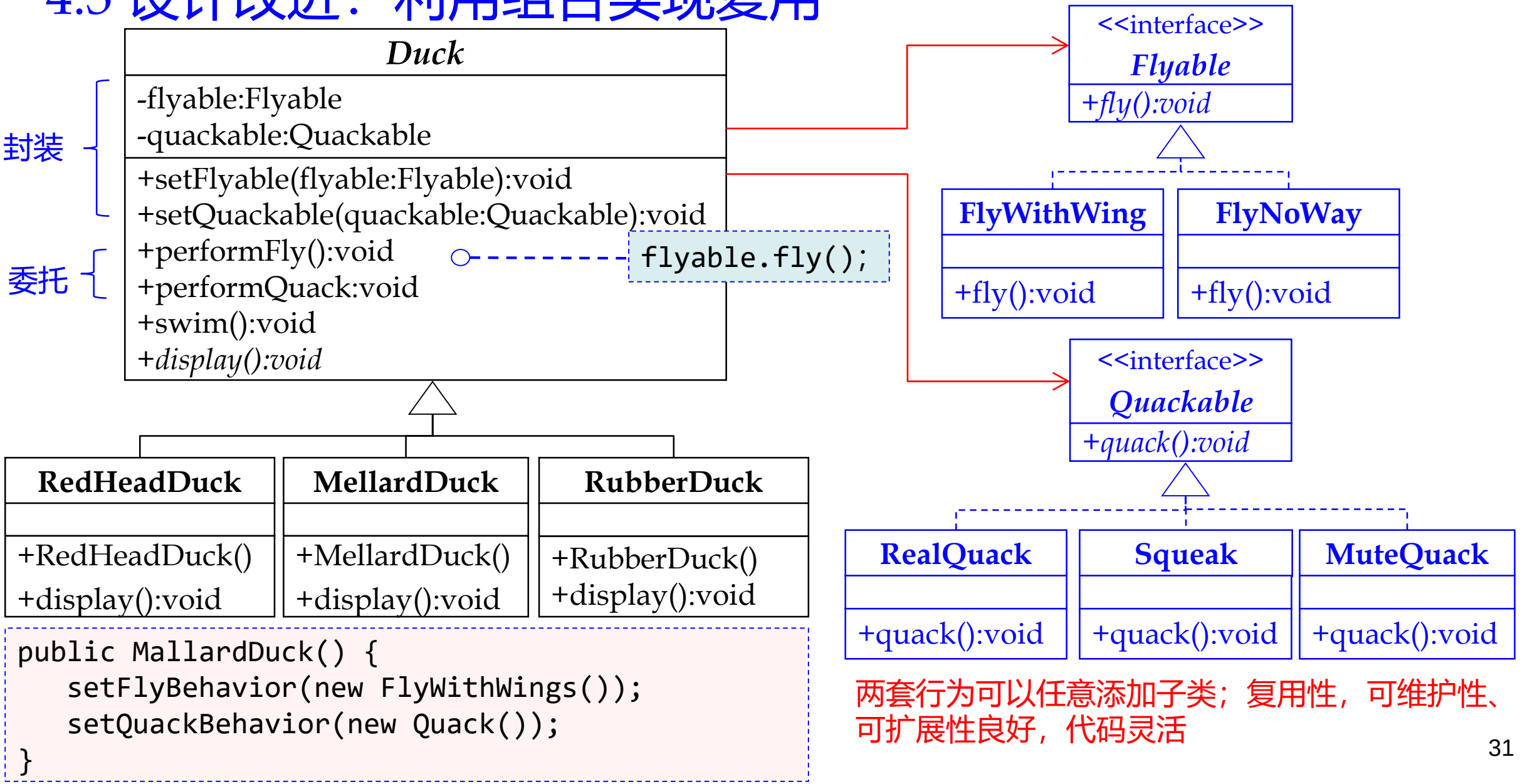




在 Duck 类中保留不变部分，让所有子类继承，达到代码复用的目的。

此时分离了变化点，并将变化点封装成接口，也有接口实现类，有了可复用的代码的基础。

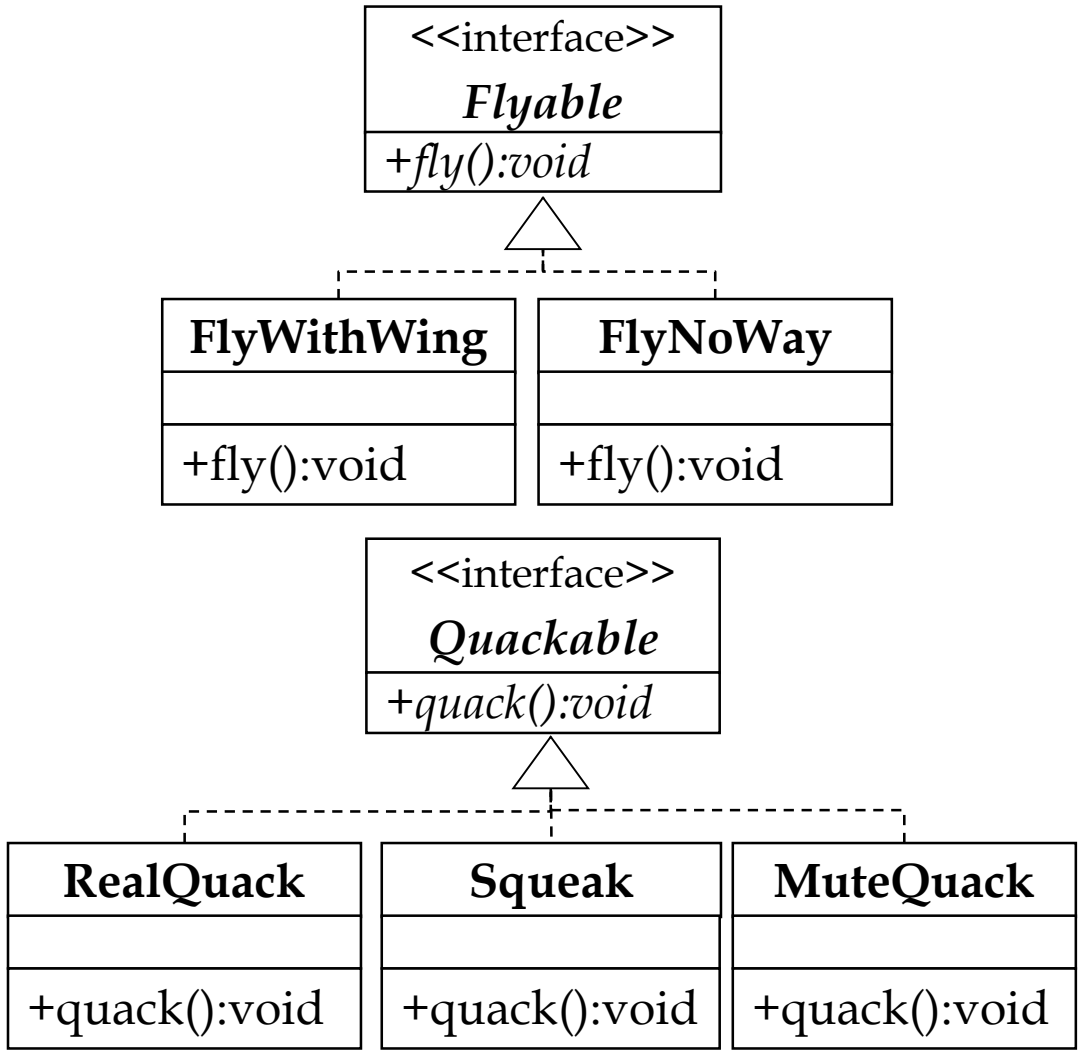
4.3 设计改进：利用组合实现复用




```
public MallardDuck() {
    setFlyBehavior(new FlyWithWings());
    setQuackBehavior(new Quack());
}
```

两套行为可以任意添加子类；复用性，可维护性、可扩展性良好，代码灵活

4.4 利用组合实现复用的代码实现



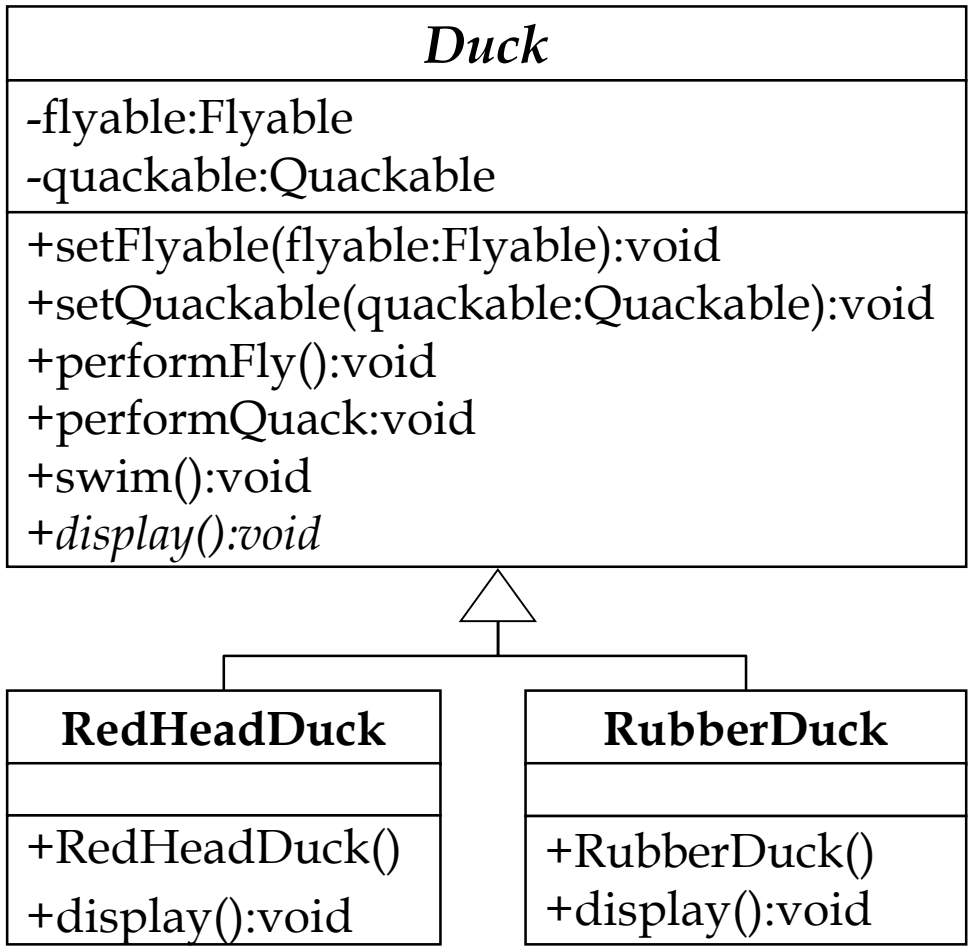

```java
public interface Flyable {
    void fly();
}
```

```java
public class FlyNoWay implements Flyable {
    public void fly() {
        System.out.println("I can't fly!");
    }
}
```

```java
public interface Quackable {
    void quack();
}
```

```java
public class Squeak implements Quackable {
    public void quack() {
        System.out.println("Squeak");
    }
}
```

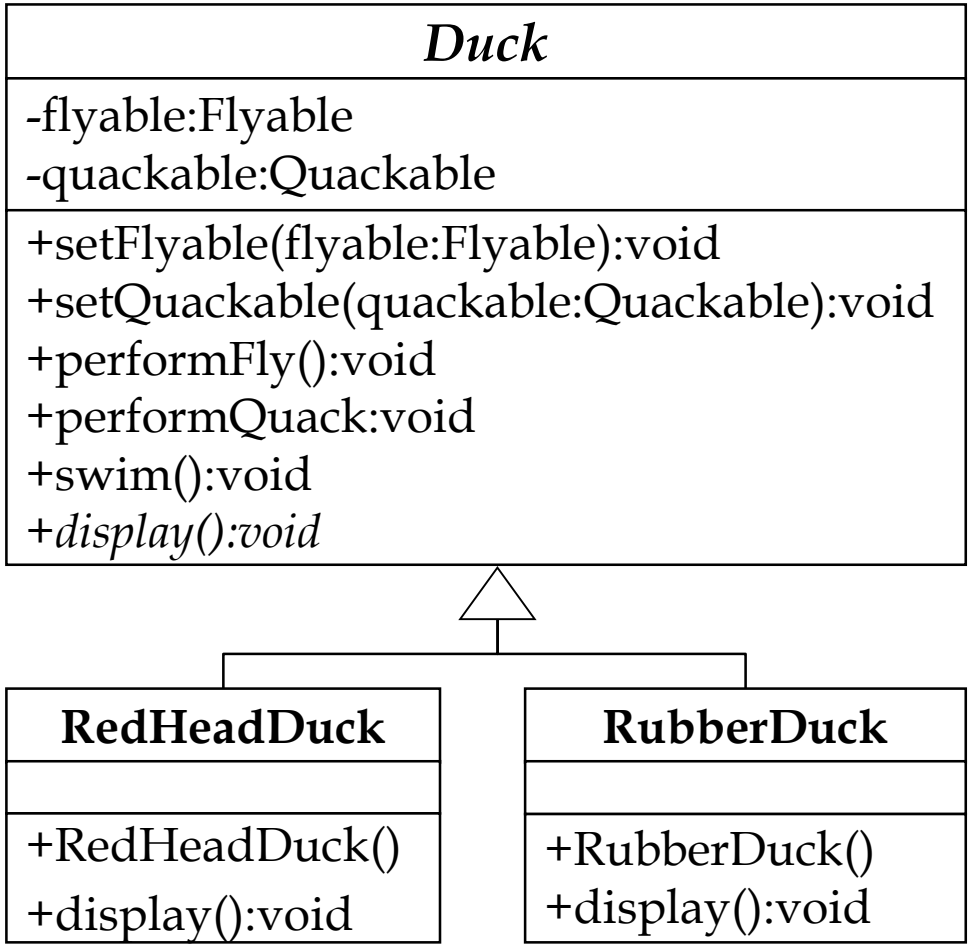
4.4 利用组合实现复用的代码实现




```
public abstract class Duck {
    private Flyable flyable;        //超类中并没有赋值
    private Quackable quackable;  //多态, 面向接口编程
    public void setQuackable(Quackable quackable) {
        this.quackable = quackable;
    }
    public void setFlyable(Flyable flyable) {
        this.flyable = flyable;
    }
    public void performFly() {
        flyable.fly();
    }
    public void performQuack() {
        quackable.quack();
    }
    public void swim() {
        System.out.println("All ducks swim");
    }
    abstract void display();
}
```

委托

4.4 利用组合实现复用的代码实现




```java
public class RedHeadDuck extends Duck {
    public RedHeadDuck() {
        setFlyable(new FlyWithWings());
        setQuackable(new RealQuack());
    }
    public void display() {
        System.out.println("I'm Red Headed duck");
    }
}
```

```java
public class RubberDuck extends Duck {
    public RubberDuck() {
        setFlyable(new FlyNoWay());
        setQuackable(new Squeak());
    }
    public void display() {
        System.out.println("I'm a rubber duckie");
    }
}
```

4.4 利用组合实现复用的代码实现

```
public class DuckTest {
    public static void main(String[] args) {

        Duck duck = new MallardDuck();
        System.out.println("MallardDuck's behavior:");
        duck.performQuack();
        duck.performFly();
        duck.swim();
        duck.display();

        System.out.println("========================");

        duck  = new RubberDuck();
        System.out.println("RubberDuck's behavior:");

        duck.performQuack();
        duck.performFly();
        duck.swim();
        duck.display();
    }
}
```

```
MallardDuck's behavior:
Quack
I'm flying!!
All ducks swim
Mallard duck
========================
RubberDuck's behavior:
Squeak
I can't fly
All ducks swim
I'm a rubber duckie
```

4.4 利用组合实现复用的代码实现

```
public class DuckTest {
    public static void main(String[] args) {

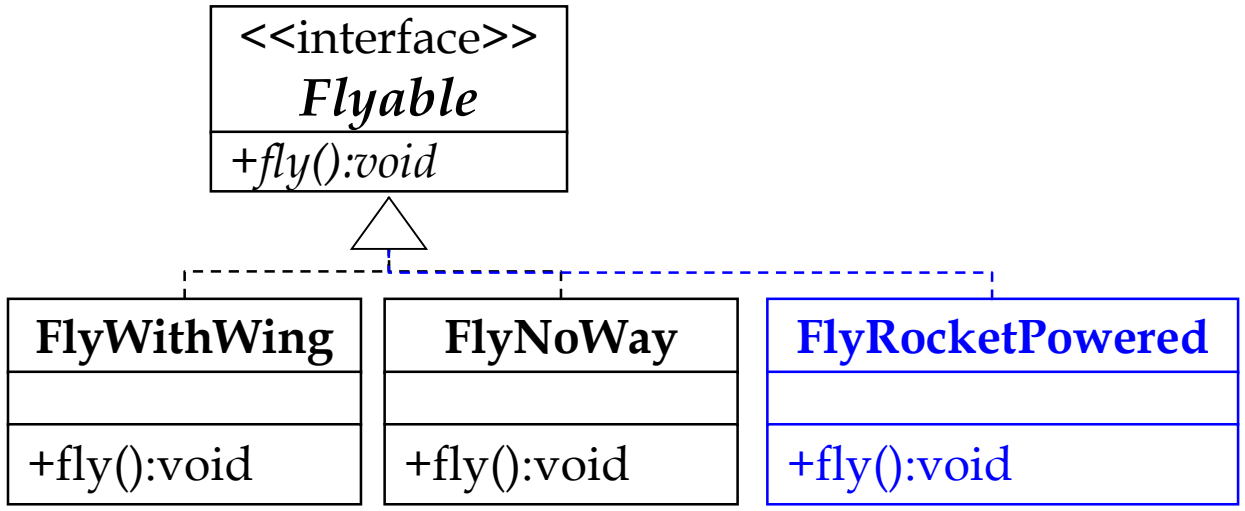
        Duck duck = new MallardDuck();                    多态, 面向接口编程
        System.out.println("MallardDuck's behavior:");
        duck.performQuack();
        duck.performFly();                                委托ok

        duck.swim();                                      复用ok

        duck.display();

        System.out.println("========================");
        duck = new RubberDuck();                          多态, 面向接口编程
        System.out.println("RubberDuck's behavior:");
        duck.performQuack();
        duck.performFly();
        duck.swim();
        duck.display();
    }
}
```

4.5 在运行过程中，动态设置 duck 对象的行为




```
public class FlyRocketPowered implements Flyable {
    public void fly() {
        System.out.println("flying with rocket");
    }
}
```

```
public class DuckTest {
    public static void main(String[] args) {

        Duck duck  = new RubberDuck();
        System.out.println("Rubber's behavior:");
        duck.swim();
        duck.display();
        duck.performQuack();

        duck.performFly();
        System.out.println("to be changed....");
        duck.setFlyable(new FlyRocketPowered());
        duck.performFly();
    }
}
```

```
I can't fly
fly to be changed....
I'm flying with a rocket
```

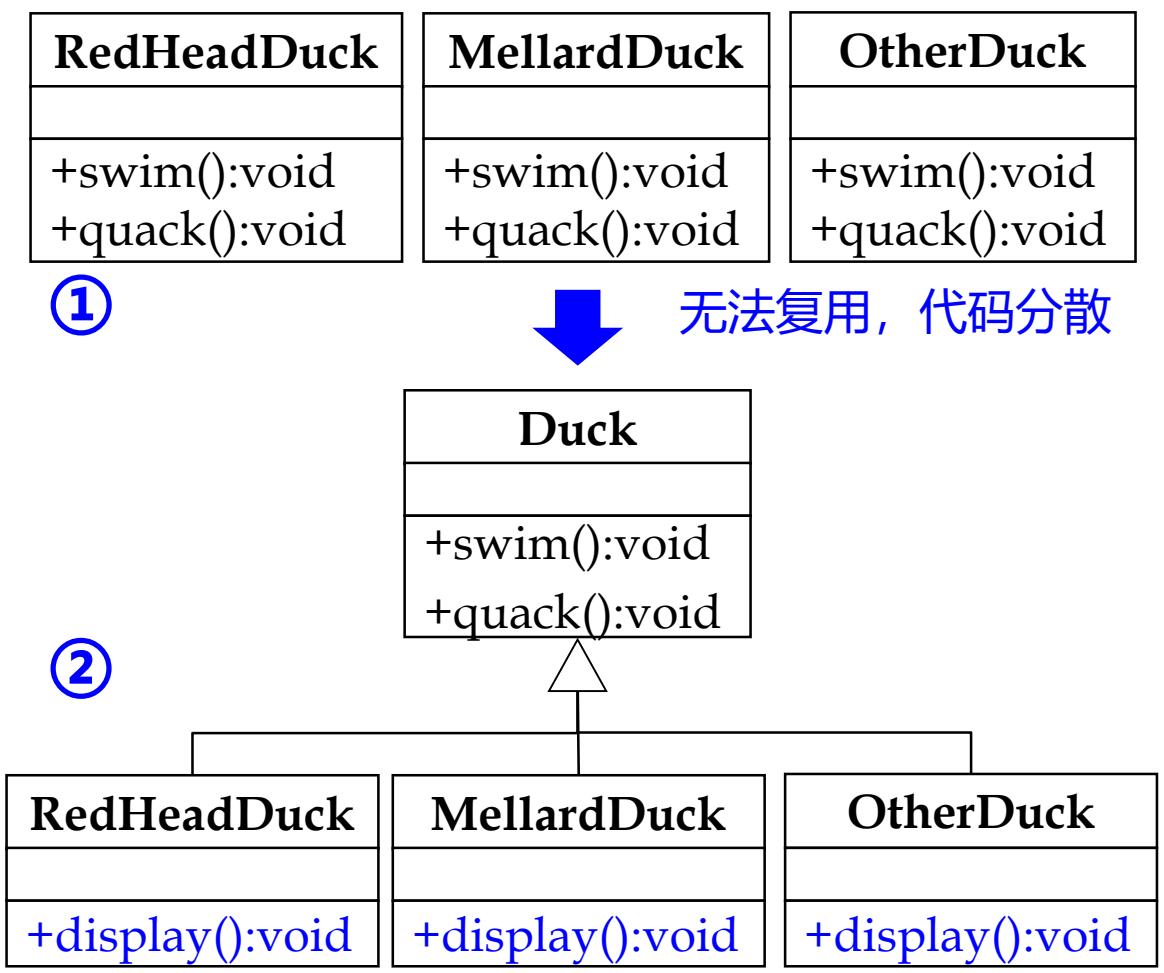
变更前：fly行为由构造函数初始化 Duck duck = new RubberDuck();

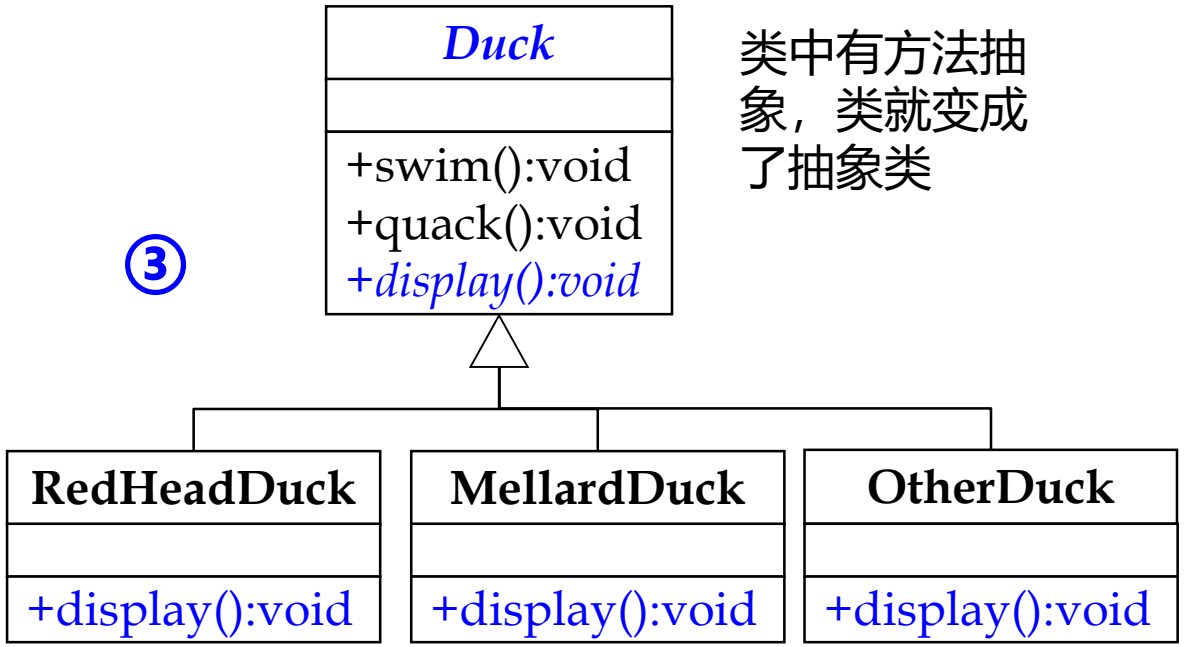
动态设定行为： duck.setFlyable(new FlyRocketPowered());

5. 设计过程复盘与讨论

Review and discussion of the design process

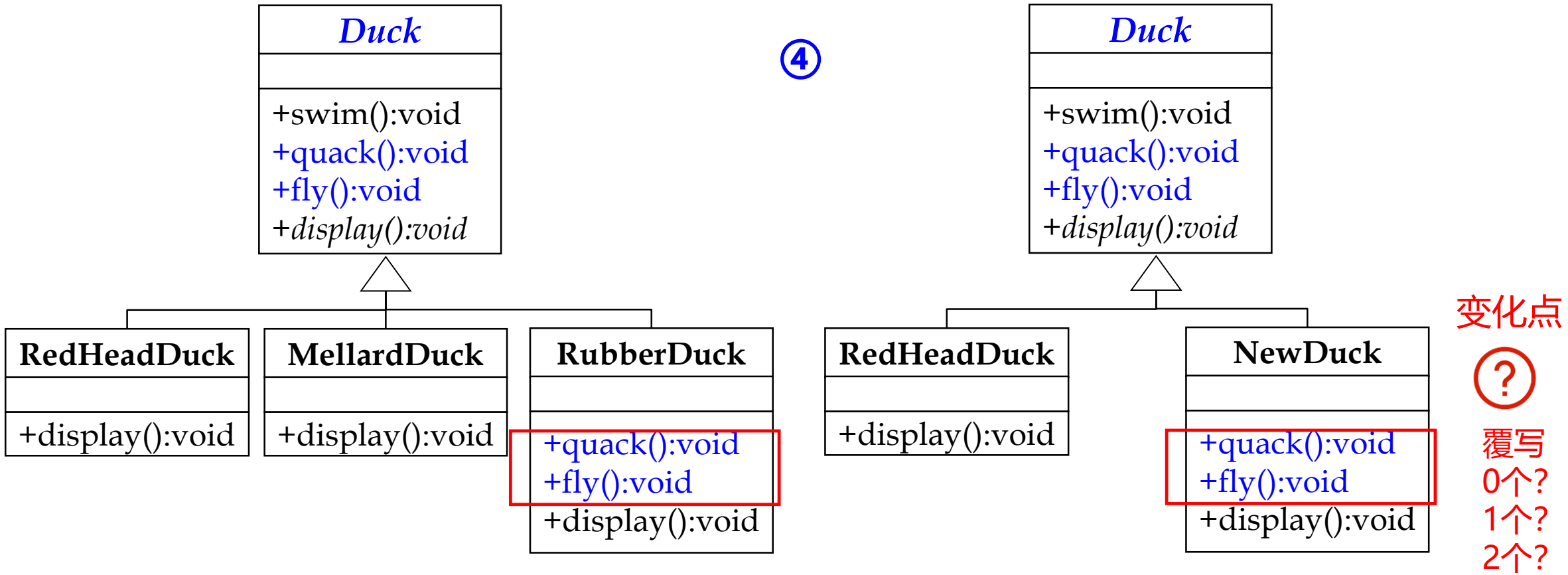
5.1 设计过程复盘 – 初步设计





初步设计是基本的设计思维：能够复用，也进行了抽象，支持扩展。但因为采用继承实现，因此无法避免继承存在的问题。

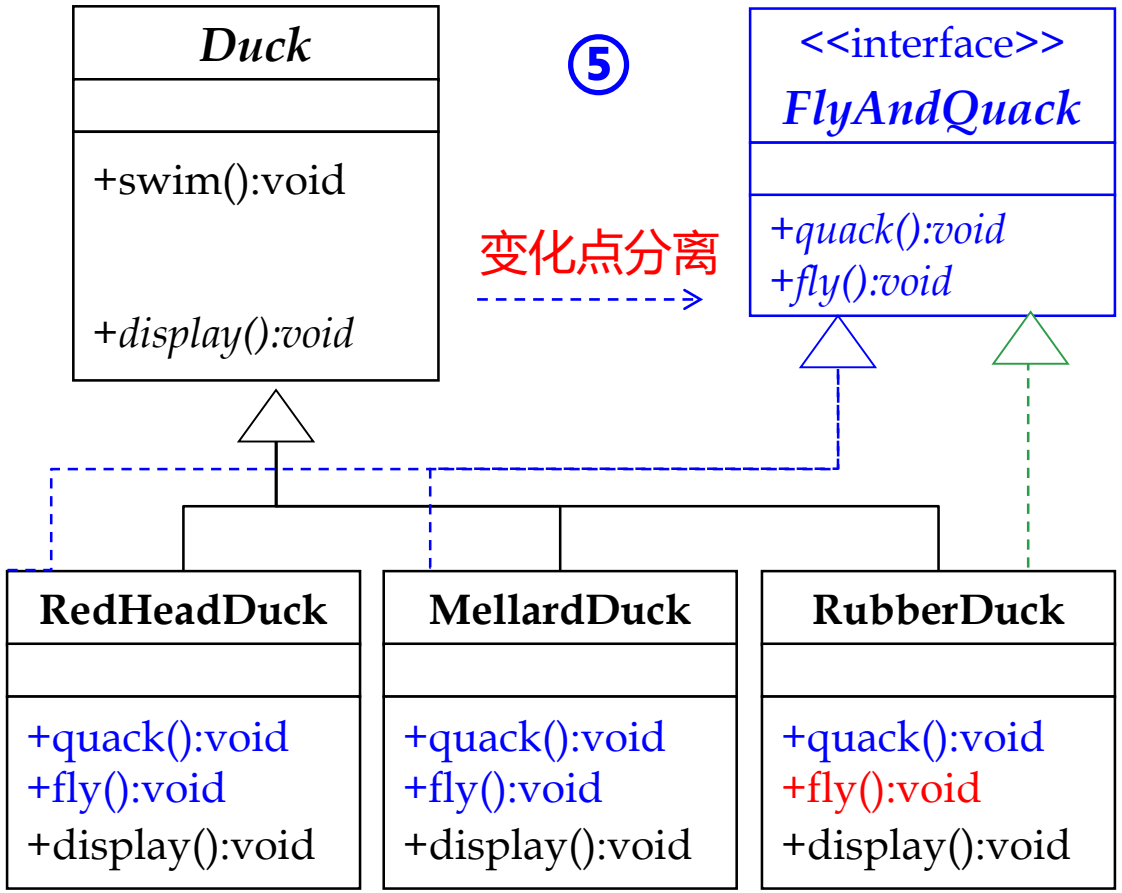
5.1 设计过程复盘 – 通过覆写改变子类行为



继承结构中，子类和父类高度耦合，修改父类的代码，会直接影响到子类

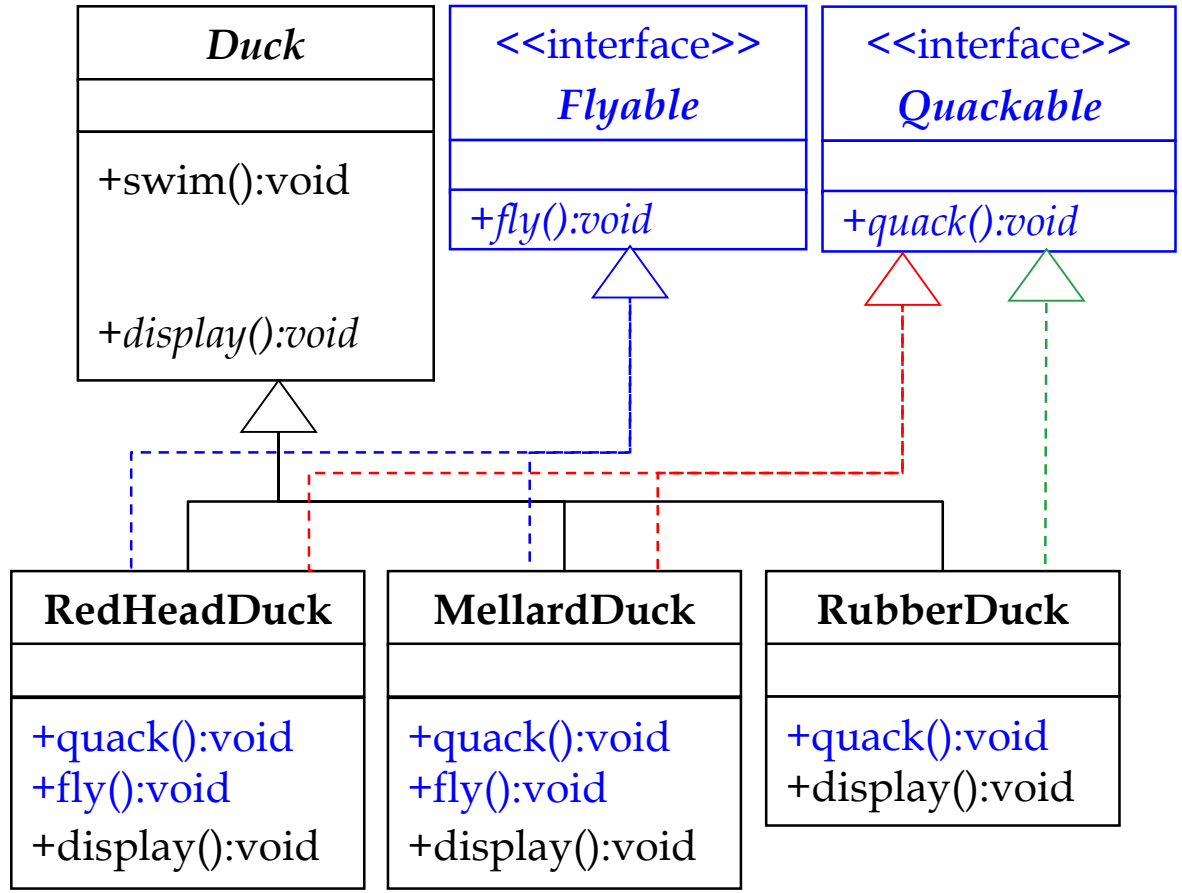
结论：局部变更影响的层面不一定仅限于局部。

5.1 设计过程复盘 – 分离变化点



方案1：quack() 和 fly() 分离到一个接口

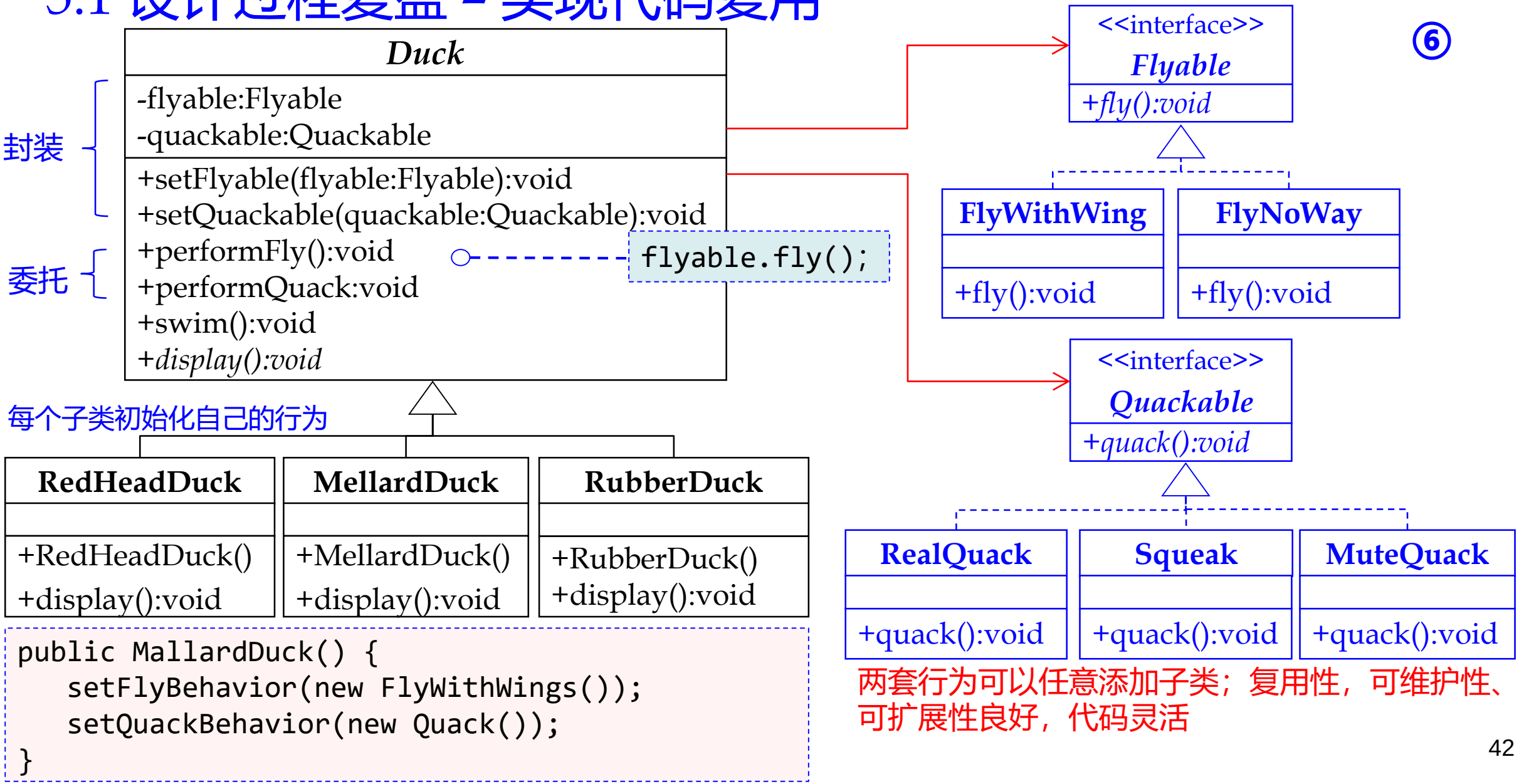
实现 FlyAndQuack 接口类的子类必须 会飞 + 会叫，无论是不是具有这两种行为



方案2：quack() 和 fly() 分离到不同的接口

通过实现不同接口类满足：有的会飞，有的会叫，有的会飞+会叫；

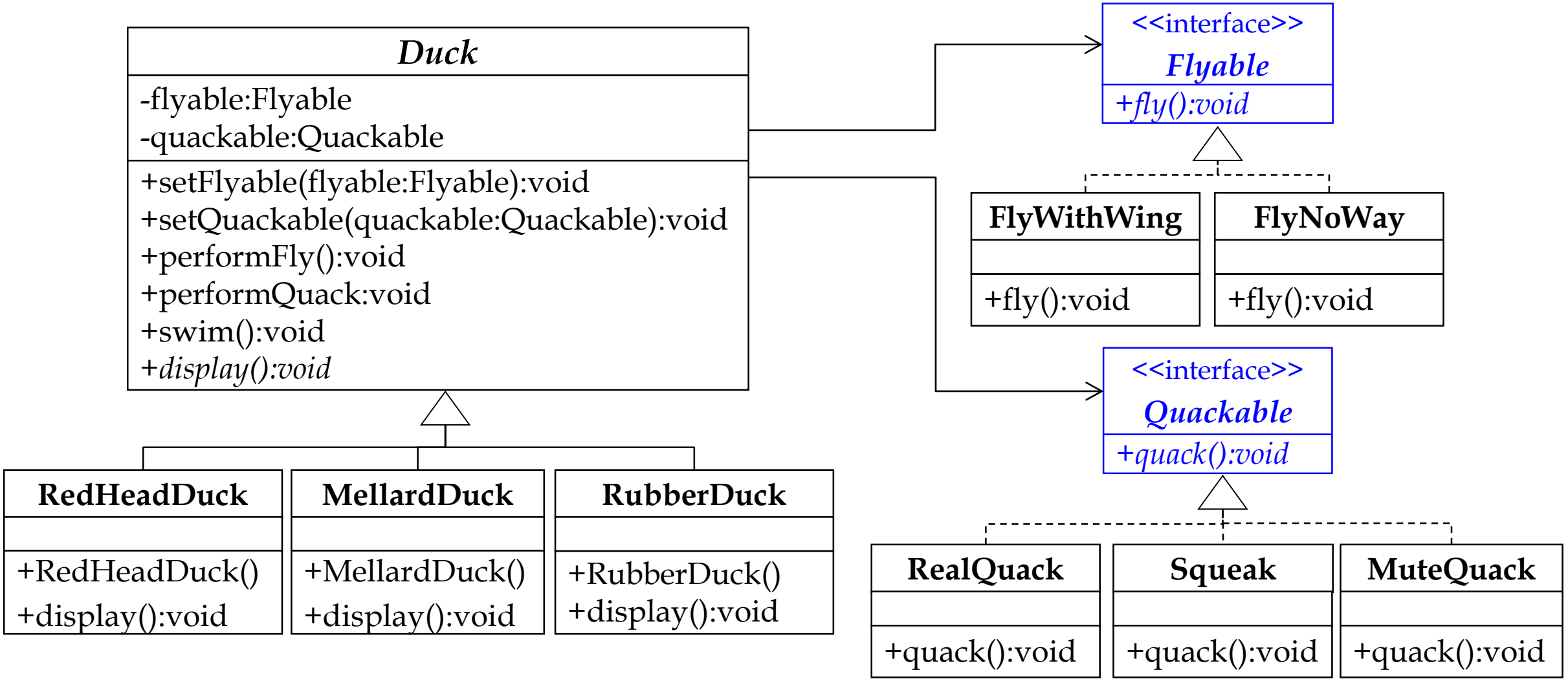
5.1 设计过程复盘 – 实现代码复用




```
public MallardDuck() {
    setFlyBehavior(new FlyWithWings());
    setQuackBehavior(new Quack());
}
```

两套行为可以任意添加子类；复用性，可维护性、可扩展性良好，代码灵活

5.2 讨论：如果 Duck 对象的行为类型增加，如何改进设计？

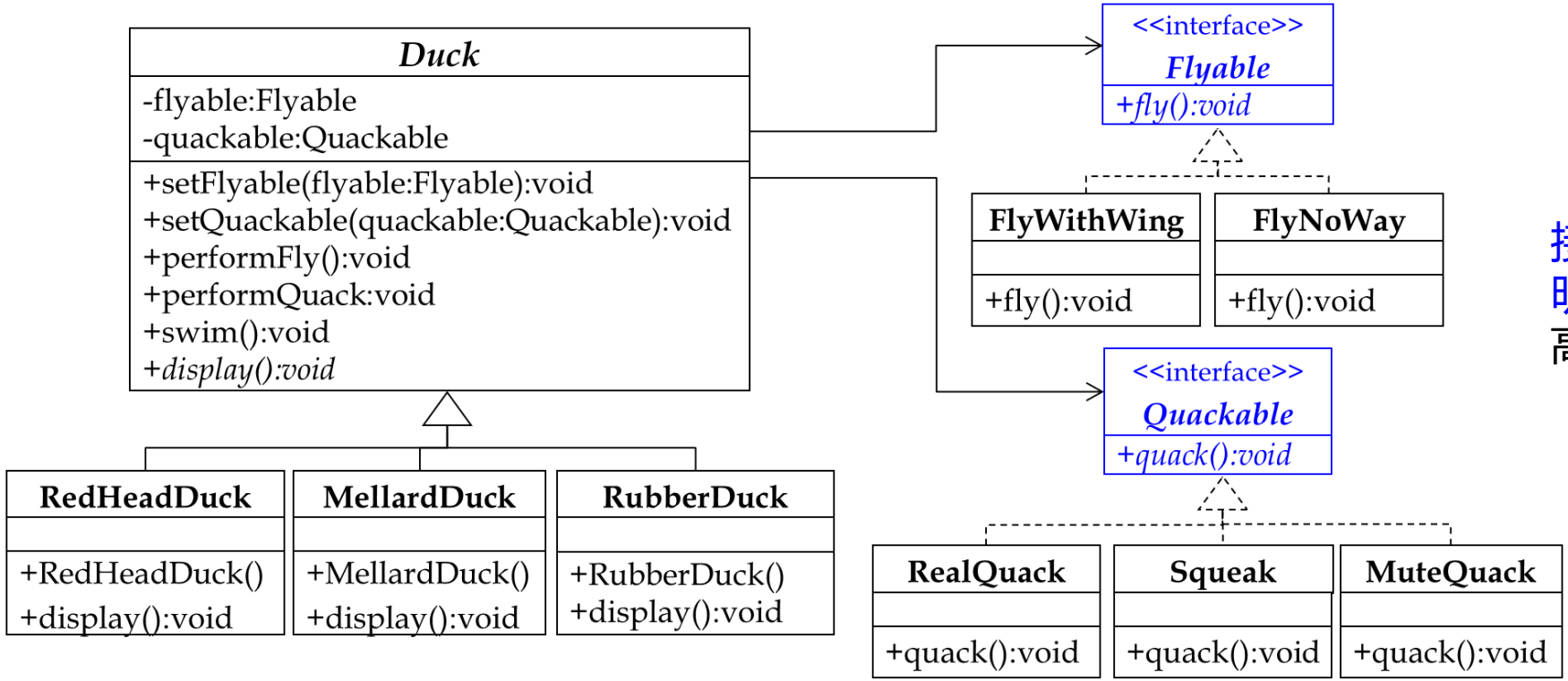


如果 Duck 对象，再增加一套行为，例如 run，是否还满足良好的可维护性？如何解决，留给大家思考。

5.3 讨论：接口类与抽象类使用时机

抽象类是对成员变量和方法的抽象，强调“是什么”，是为了解决代码复用问题。

接口类仅仅是对方法的抽象，强调“具有什么能力”，是为了解决抽象而非代码复用。



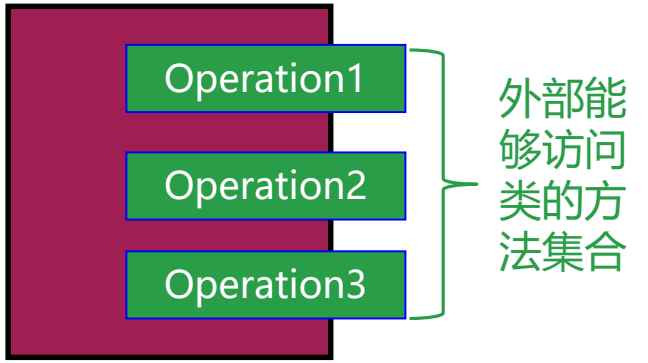
接口类隔离接口方法的声明和方法的代码实现，提高代码的扩展性。

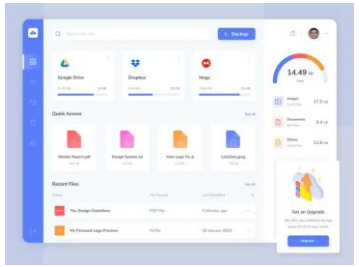
抽象类是自下而上的设计思路。先有子类的代码重复，然后再抽象成上层的父类

接口类是自上而下的设计思路。一般都是先设计接口，再去考虑具体的实现。

5.4 讨论：为什么基于接口而非实现编程

- 接口的概念在 OOP 中存在多种语义：





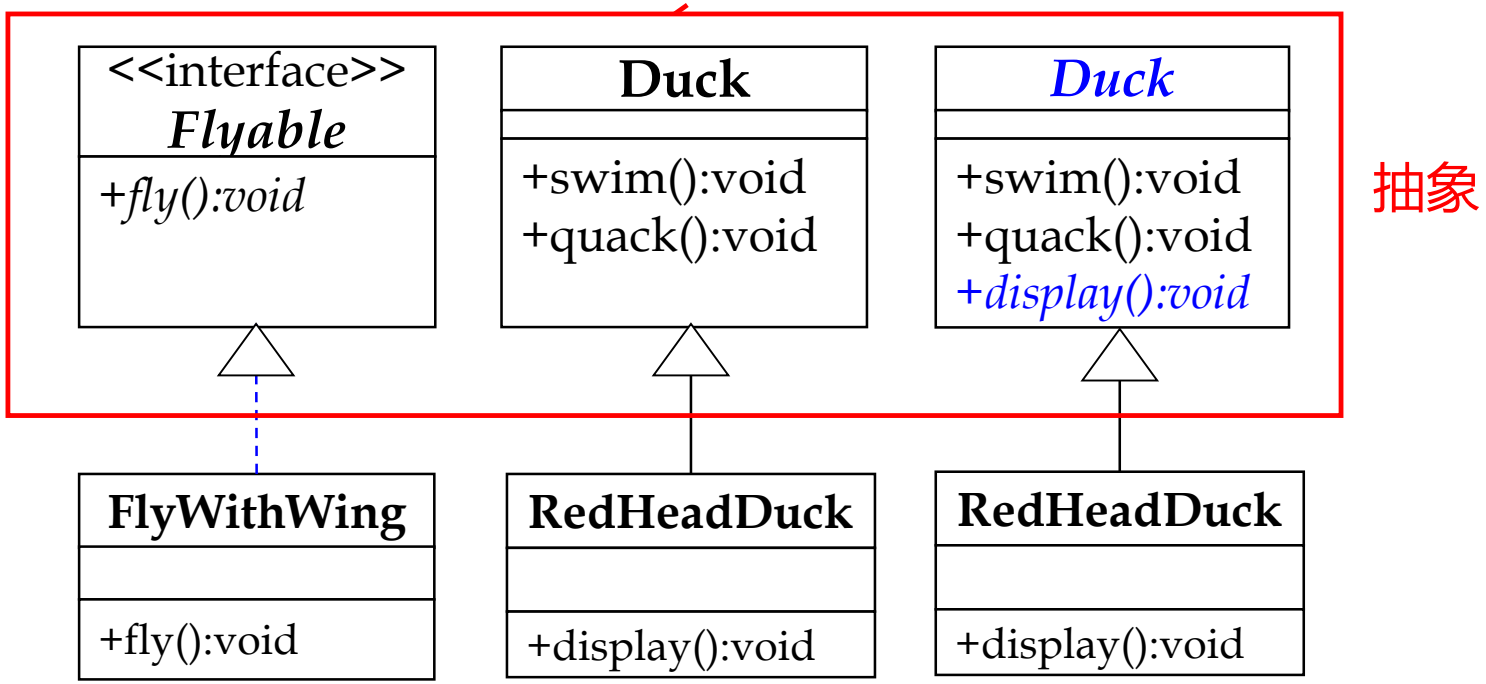
人机接口（界面）

一组 API 接口集合：



前端接口　　DB接口

“基于接口而非实现编程”这条原则中的“接口”，是指顶层类或者接口



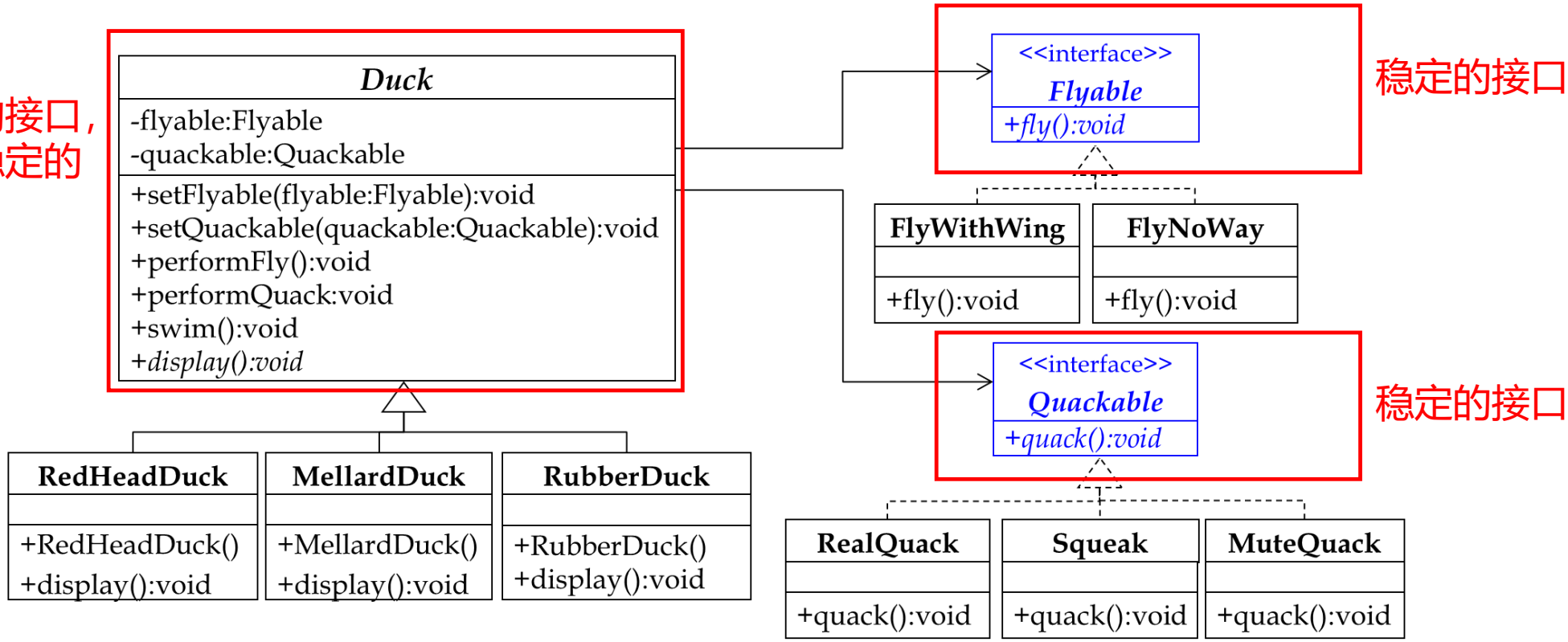
抽象

基于接口变成抽象类和Interface (本课程中称为接口类)，在某些场合下父类也可以认为是接口

5.4 讨论：为什么基于接口而非实现编程

- 基于接口编程，而非实现编程：将接口和实现相分离，封装不稳定的实现，暴露稳定的接口

设计良好的接口，都应该是稳定的

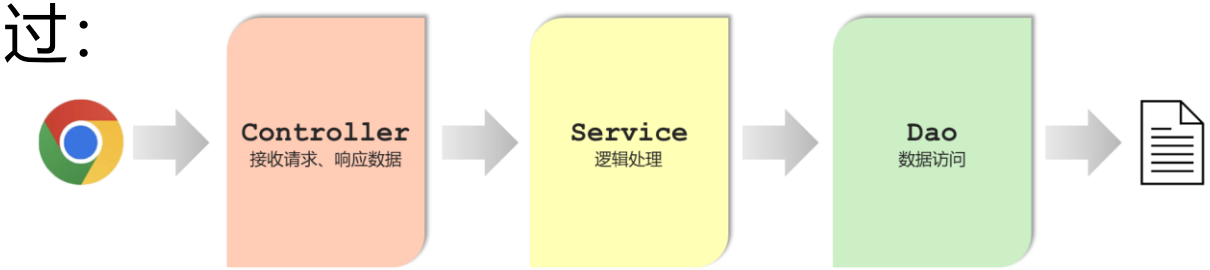


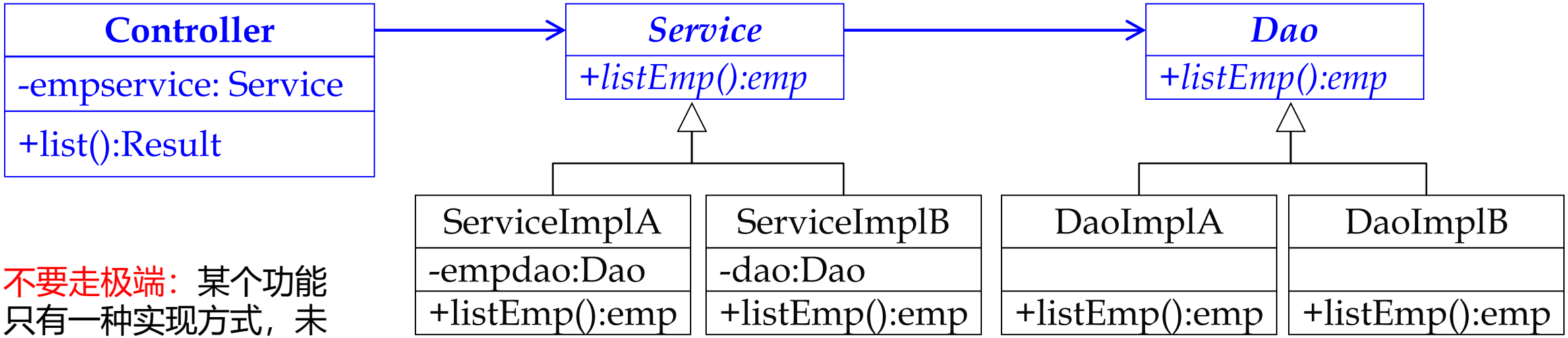
越抽象、越顶层、越脱离具体某一实现的设计，越能提高代码的灵活性，越能应对未来的需求变化。

好的代码设计，不仅能应对当下的需求，而且在将来需求发生变化的时候，仍然能够在不破坏原有代码设计的情况下灵活应对。

5.4 讨论：为什么基于接口而非实现编程

- 在2025春季学期Web课程中已经应用过：



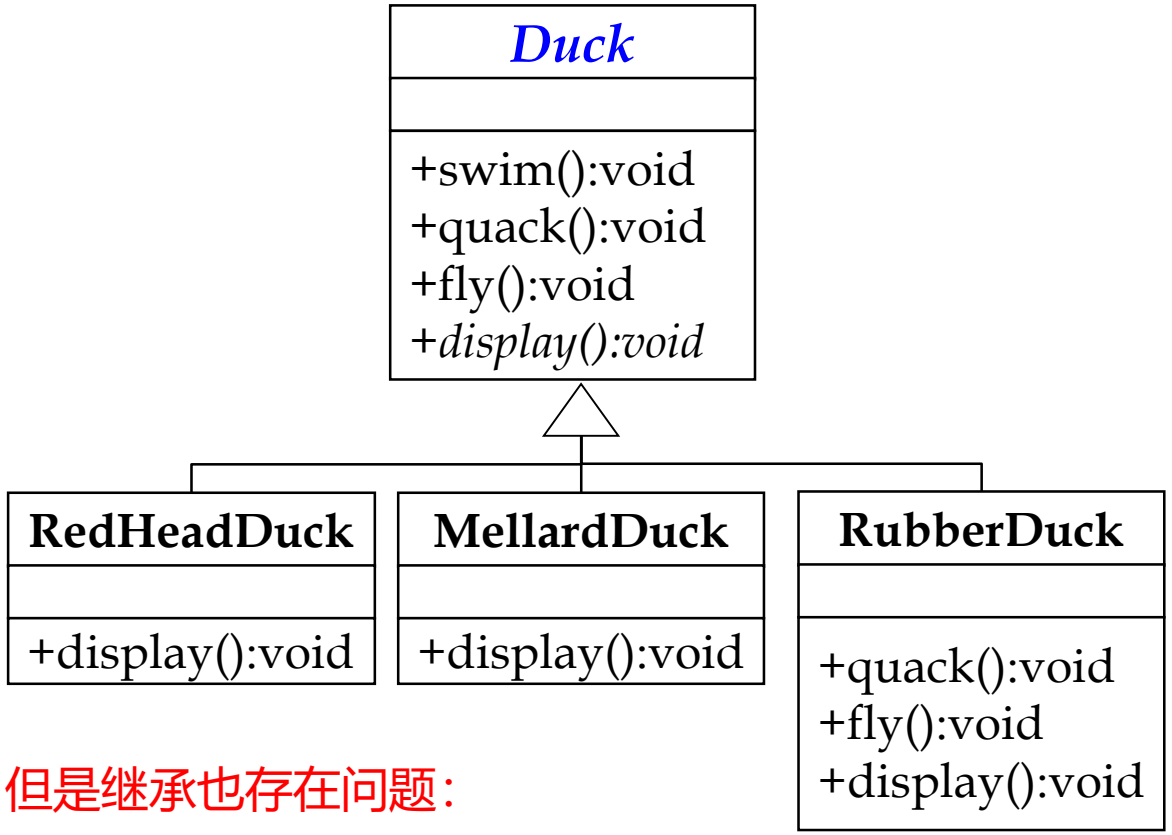


不要走极端：某个功能只有一种实现方式，未来也不可能被其他实现方式替换，那就没有必要为其设计接口，也没有必要基于接口编程

上游系统（例如Controller）面向接口而非实现编程，不依赖不稳定的实现细节，当实现发生变化的时候，上游系统的代码基本上不需要做改动，以此来降低代码间的耦合性，提高代码的扩展性。

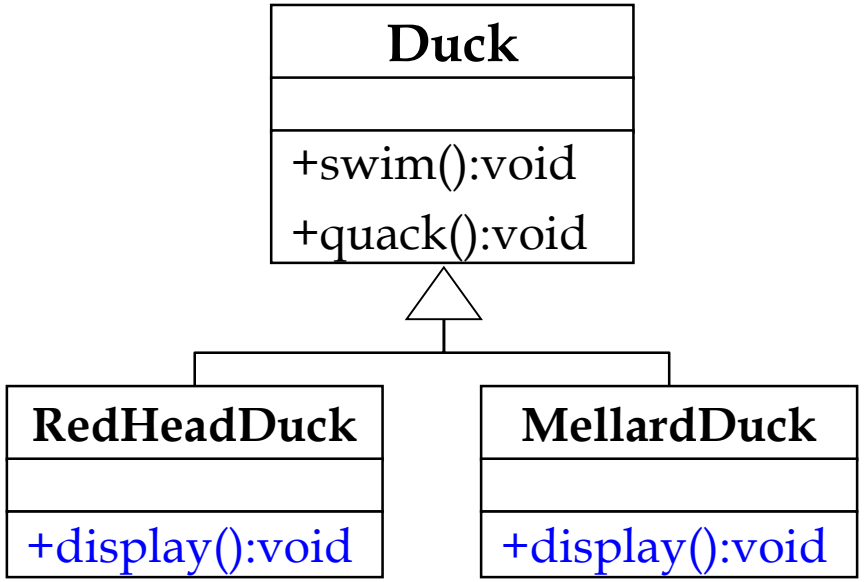
5.5 讨论：为何说要多用组合少用继承?

继承主要有三个作用：1.表示 is-a 关系；2.支持多态特性；3.代码复用。



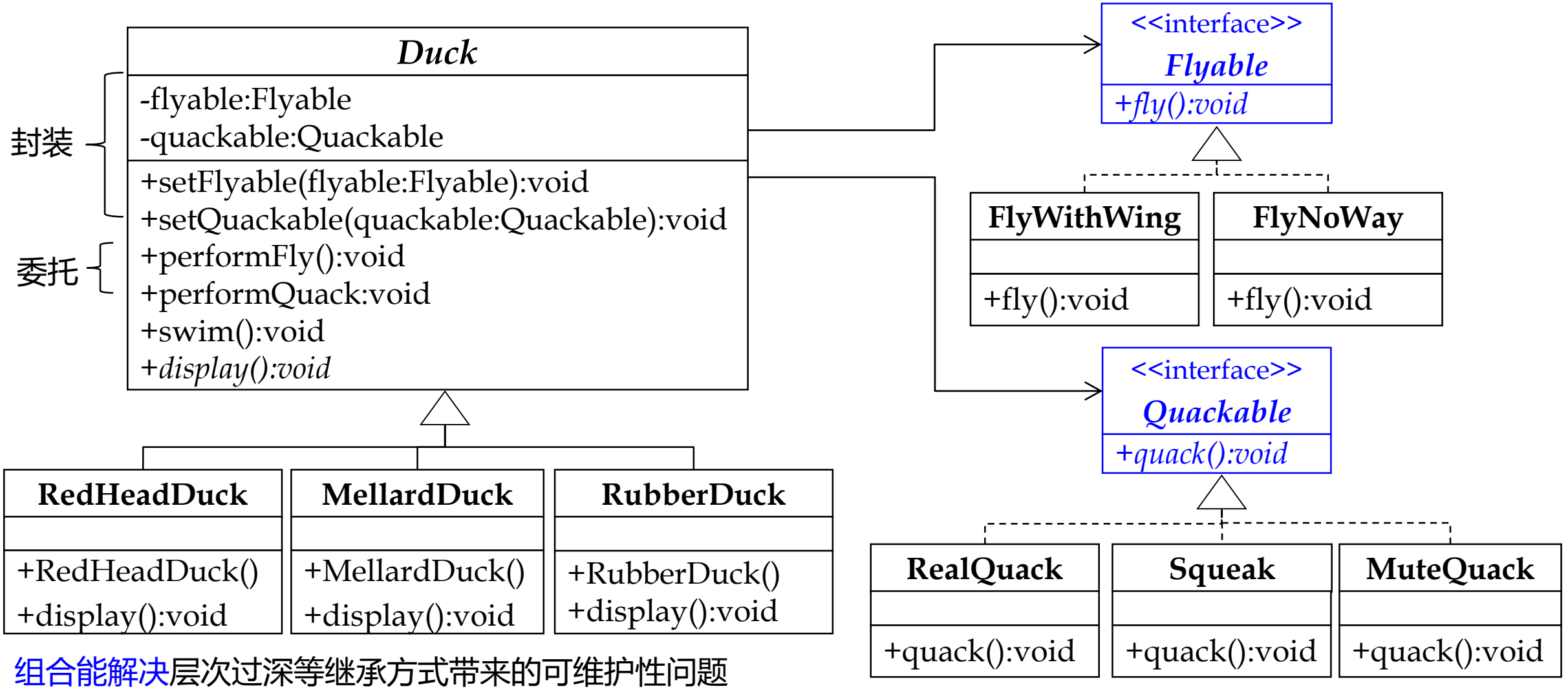
但是继承也存在问题：

- 继承存在的问题1：继承层次过深、继承关系过于复杂会影响到代码的可读性和可维护性
- 继承存在的问题2：超类中的变更影响所有的子类
- 继承存在的问题3：子类对父类可以扩展，如果子类中添加了父类中没有的方法，多态特性消失，例如：




```
Duck duck = new MallardDuck();
duck.display();    // 编译错误
```

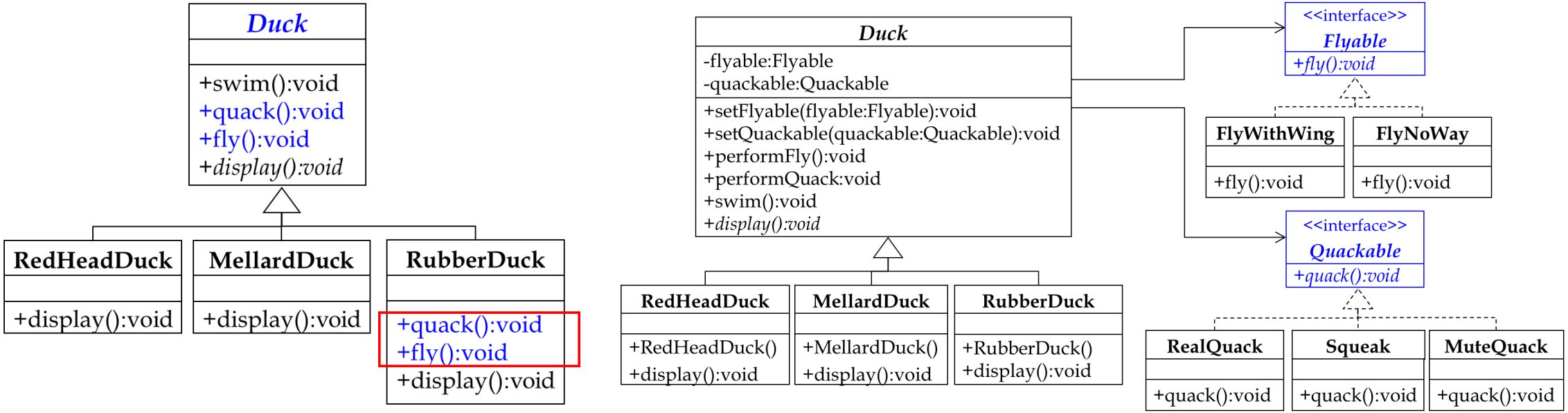
5.5 讨论：为何说要多用组合少用继承?



组合能解决层次过深等继承方式带来的可维护性问题

5.5 讨论：为何说要多用组合少用继承?

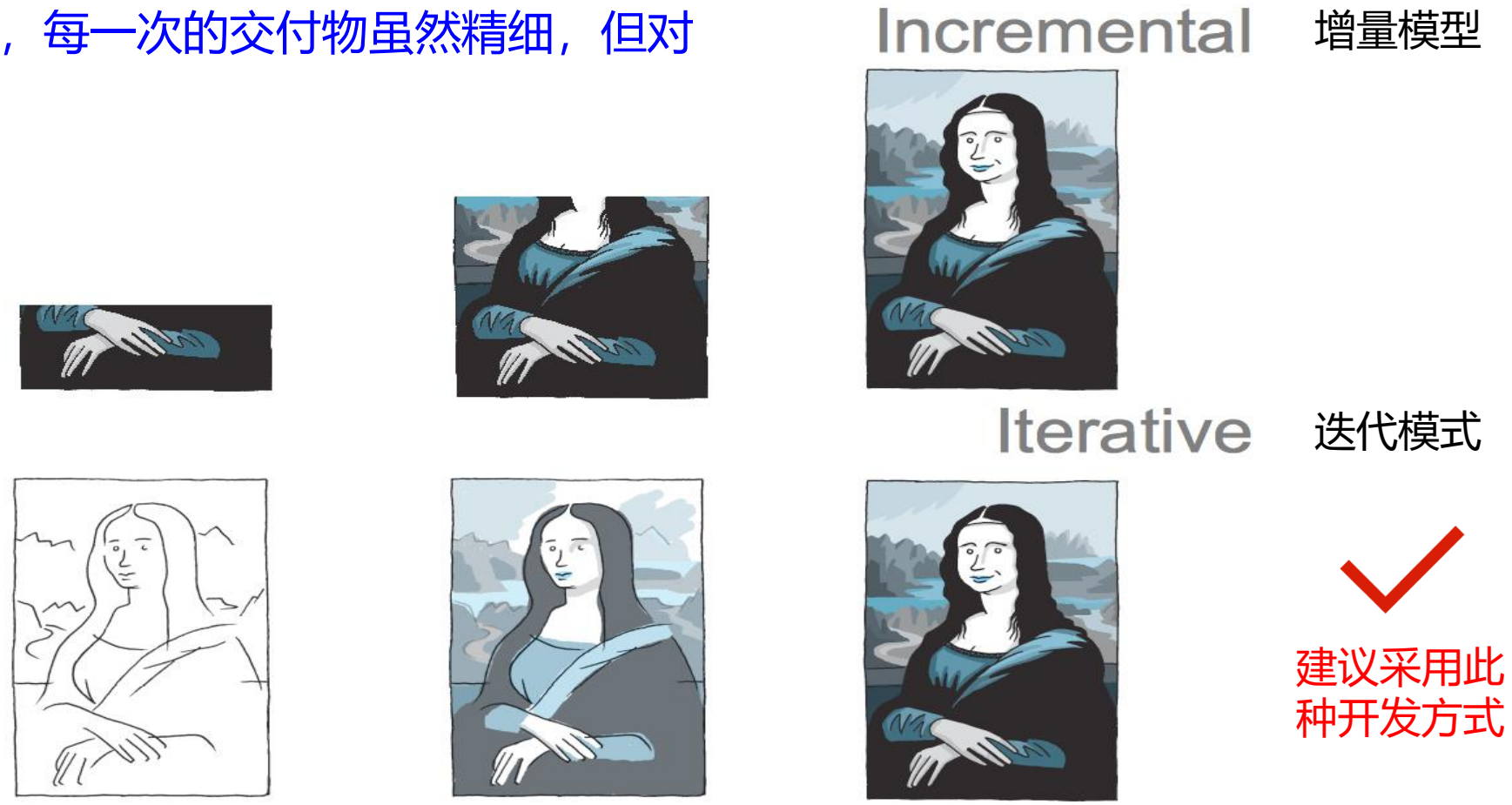
- 尽管鼓励多用组合少用继承，但组合也并不是完美，继承也并非一无是处。组合意味着要做更细粒度的类的拆分，要定义更多的类和接口，在一定程度上增加了代码的复杂度和维护成。



- 如果类之间的继承结构稳定，继承层次比较浅（最多有两层继承关系），继承关系不复杂，就可以大胆地使用继承。

5.6 讨论：软件开发过程的 增量模型 vs 迭代模型

增量模型：增量开发中，每一次的交付物虽然精细，但对用户来说不可用



迭代模型：每一次的交付物都是完整、用户可用的。因此本案例通过迭代思想，多次迭代重构设计。

The end

November 8, 2025